
Generación de árboles de comportamiento de
personajes no jugador mediante grandes
modelos de lenguaje
Generation of Non-Player Character Behavior
Trees Using Large Language Models



Trabajo de Fin de Máster
Curso 2025–2026

Autor

Pablo Sánchez Martín

Directores

Carlos León Aznar, Ismael Sagredo Olivenza

Máster en Inteligencia Artificial

Facultad de Informática

Universidad Complutense de Madrid

Generación de árboles de comportamiento
de personajes no jugador mediante
grandes modelos de lenguaje
Generation of Non-Player Character
Behavior Trees Using Large Language
Models

Trabajo de Fin de Máster en Máster de Inteligencia Artificial

Autor

Pablo Sánchez Martín

Director

Carlos León Aznar, Ismael Sagredo Olivenza

Convocatoria: *Septiembre 2026*

Calificación: *8.0*

Máster en Inteligencia Artificial

Facultad de Informática

Universidad Complutense de Madrid

19 de septiembre de 2026

Dedicatoria

*A toda la gente que quiero,
a los que me han apoyado y a los que me han
hecho crecer.
A todos los que han estado a mi lado.*

Agradecimientos

A mi familia: mi madre, mi hermana, su recién marido, mis tíos y mis primos por haber soportado otro año más de mis estreses académicos. A David y a María, por estar todos los días a mi lado y haber hecho que ir a clase haya sido tan divertido. A mis amigos de la universidad, por soportarme todos los días, escucharme y hacer más amenos mis días. A mis amigos del instituto y del colegio, por seguir a mi lado apoyándome y animándome cuando lo necesito. A aquellos profesores que, un año más, han estado a mi lado ayudándome a evolucionar, soportándome y dándome su ayuda en los momentos que lo he necesitado. En especial, a mis tutores, Carlos e Isma, por estar conmigo en todo y ofrecerme desde un principio un tiempo que no les sobra.

Resumen

Generación de árboles de comportamiento de personajes no jugador mediante grandes modelos de lenguaje

Este estudio se centra en la creación de una herramienta que, mediante grandes modelos de lenguaje, sea capaz de generar árboles de comportamiento (BTs) para personajes no jugador (NPCs) que sirvan como base con el fin de aliviar la carga de trabajo de los diseñadores.

Para ello se ha elegido el motor de videojuegos Unreal Engine 5 para implementar una serie de acciones y condicionales que servirán como nodos para los BTs generados, así como para diseñar una serie de escenarios que sirvan como entornos de pruebas en los que se puedan ejecutar los BTs. Una vez se han contado con una serie de nodos y entornos de ejecución, se ha procedido al diseño de la herramienta. Finalmente se decidió por una implementación en la que el modelo “Mistral-large-3” procesase el comportamiento escrito en lenguaje natural en un pseudocódigo que se traducía a una estructura de árbol en formato JSON, mientras que un segundo modelo más pequeño, el modelo “llama3:8B” se dedicaba a la asignación de variables de la Blackboard (módulo de memoria del BT). Una vez se genera el BT se han implementado dos formas de validación automática y posible corrección, un autómata finito no determinista (NFA) que valida si el orden de los nodos respeta posibles dependencias entre ellos, y un validador que, al terminar la ejecución, verifica si se han cumplido ciertos objetivos planteados para el comportamiento. Al tener el sistema planteado se han diseñado unas pruebas de usuario para comprobar si la herramienta es intuitiva y genera BTs de una calidad similar a los que crea un humano en un tiempo inferior. Las pruebas se han realizado con una muestra de $N = 3$ usuarios, por lo que no se pueden tomar conclusiones definitivas. Sin embargo, se ha observado que los usuarios encontraban la herramienta fácil de usar (con una puntuación SUS de 90.83/100) y que generaban los BTs en un tiempo inferior a los humanos con una calidad ligeramente inferior, cumpliendo así con el objetivo de la herramienta.

Palabras clave

IA, Árboles de comportamiento, Comportamiento de NPCs, Procesamiento del lenguaje natural, LLM

Abstract

Generation of Non-Player Character Behavior Trees Using Large Language Models

This study focuses on creating a tool that, using large language models, is capable of generating behavior trees (BTs) for non-player characters (NPCs) to serve as a foundation for reducing the workload of designers.

To this end, the Unreal Engine 5 video game engine was chosen to implement a series of actions and conditionals that will serve as nodes for the generated BTs, as well as to design a series of scenarios that will serve as test environments in which the BTs can be executed. Once a set of nodes and execution environments was in place, the tool was designed. Finally, an implementation was chosen in which the “Mistral-large-3” model processed the behavior written in natural language into pseudocode that was translated into a tree structure in JSON format, while a second, smaller model, the “llama3:8B” model, was dedicated to assigning variables in the Blackboard (the BT’s memory module). Once the BT is generated, two methods for automatic validation and potential correction have been implemented: a non-deterministic finite automaton (NFA) that validates whether the order of the nodes respects any dependencies between them, and a validator that, upon completion of execution, verifies whether certain objectives set for the behavior have been met. Once the system was set up, user tests were designed to verify whether the tool is intuitive and generates BTs of a quality similar to those created by a human in less time. The tests were conducted with a sample of $N = 3$ users, so no definitive conclusions can be drawn. However, it was observed that users found the tool easy to use (with an SUS score of 90.83/100) and that it generated BTs in less time than humans, albeit with slightly lower quality, thus meeting the tool’s objective.

Keywords

AI, Behavior Trees, NPC Behavior, Natural Language Processing, LLM

Índice

1. Introducción	1
1.1. Motivación	2
1.2. Objetivos	3
1.3. Plan de trabajo	4
1.4. Metodología y tecnología	4
1.5. Estructura del documento	5
1.6. Enlace al repositorio	6
1.7. Uso de Inteligencia Artificial	6
2. Estado de la Cuestión	7
2.1. Comportamientos de NPCs en videojuegos	7
2.1.1. Máquinas de estado finitas	7
2.1.2. Máquinas de estado finitas jerárquicas	8
2.1.3. Árboles de comportamiento	10
2.1.3.1. Árboles de comportamiento en Unreal Engine	11
2.2. Modelos de lenguaje	12
2.3. Generación automática de comportamientos	16
2.3.1. Generación de árboles de comportamiento previos a LLMs	16
2.3.2. Generación de árboles de comportamiento mediante LLMs en la robótica	17
2.3.3. Generación de árboles de comportamiento mediante LLMs en videojuegos	18
3. Herramienta de generación de árboles de comportamiento	21
3.1. Diseño del pipeline del modelo propuesto de generación de árboles de comportamiento	21
3.2. Descripción del entorno en un motor de videojuegos	22
3.2.1. Tareas y condicionales	24
3.2.2. Formato del árbol de comportamiento generado	25
3.3. Generación de los árboles de comportamiento	26
3.3.1. Generación de forma directa	26
3.3.2. Generación mediante fragmentación por subtareas	27

3.3.3. Generación mediante pseudocódigo	28
3.4. Validación automática de los árboles de comportamiento	34
3.4.1. Validación de dependencias	34
3.4.2. Validación de objetivos	35
3.4.3. Entornos creados para la validación	35
4. Evaluación y resultados	39
4.1. Pruebas con usuarios	39
4.2. Resultados de las pruebas de usuario	40
4.2.1. Resultados de la usabilidad de la herramienta	41
4.2.2. Resultado de la comparativa de tiempos	41
4.2.3. Resultados de la calidad y disminución de carga de trabajo . .	42
5. Discusión	47
5.1. Discusión sobre la herramienta generadora de árboles de comporta- miento	47
5.2. Discusión sobre la validación	48
5.3. Discusión de las hipótesis de las pruebas de usuario	49
5.4. Limitaciones	49
6. Conclusiones y Trabajo Futuro	51
6.1. Conclusiones	51
6.2. Trabajo Futuro	52
Introduction	53
6.3. Motivation	54
6.4. Objectives	55
6.5. Work Plan	55
6.6. Methodology and Technology	56
6.7. Document Structure	57
6.8. Link to Repository	57
6.9. Use of Artificial Intelligence	58
Conclusions and Future Work	59
6.10. Conclusions	59
6.11. Future Work	59
Bibliografía	61
A. JSON Schema para los árboles de comportamiento generados	69
B. Prompt del sistema para el LLM generador del pseudocódigo	73
C. JSON de dependencias de tareas en un árbol de comportamiento	77
D. Formulario de opiniones de la herramienta	79

Índice de figuras

2.1.	Ejemplo de un FSM para videojuegos	8
2.2.	Ejemplo de una HFSM en videojuegos	9
2.3.	Captura del sistema de animaciones de Unity Engine	9
2.4.	Ejemplo de un árbol de comportamiento	10
2.5.	Ejemplo de árbol de comportamiento de Unreal Engine	12
2.6.	Arquitectura de un modelo Transformer	13
2.7.	Arquitectura MCP	16
3.1.	Diagrama de la herramienta propuesta	22
3.2.	Entorno de desarrollo	23
3.3.	Generación mediante subtarefas	28
3.4.	Ejemplo del pseudocódigo que el LLM debe generar.	29
3.5.	Chain-of-Thought del LLM que genera el pseudocódigo.	31
3.6.	Ejemplo del input y el output del LLM que establece las variables de la Blackboard.	33
3.7.	Captura de pantalla del segundo entorno	36
3.8.	Captura de pantalla del tercer entorno	37
4.1.	Comparación de los tiempos de generación de los árboles de comportamiento	42
B.1.	Prompt del sistema para el LLM generador del pseudocódigo (primera parte)	73
B.2.	Prompt del sistema para el LLM generador del pseudocódigo (segunda parte)	74
B.3.	Prompt del sistema para el LLM generador del pseudocódigo (tercera parte)	75
B.4.	Prompt del sistema para el LLM generador del pseudocódigo (cuarta parte)	76
D.1.	Formulario de recogida de datos demográficos (primera parte)	80
D.2.	Formulario de recogida de datos demográficos (segunda parte)	81
D.3.	Formulario de recogida de datos demográficos (tercera parte)	82
D.4.	Formulario de recogida de datos demográficos (cuarta parte)	83

Índice de tablas

3.1. Modelos probados para la generación directa del JSON mediante guidance.	26
3.2. Resultados de las pruebas de validación	37
4.1. Resultados de la encuesta SUS	41
4.2. Tiempo (en segundos) de generación de cada árbol de comportamiento por cada usuario	41
4.3. Opinión de los usuarios sobre los tiempos de ejecución de la herramienta generadora de árboles de comportamiento del 1 al 5, siendo 1 una opinión desfavorable y 5 la más favorable.	42
4.4. Respuestas de los usuarios sobre la calidad de los árboles de comportamiento generados por la herramienta, donde 1 es que no están de acuerdo con la afirmación de que la calidad de los árboles generados por la herramienta es inferior y 5 es que están completamente de acuerdo.	43
4.5. Respuestas de los usuarios sobre la ejecución esperada de los árboles de comportamiento, donde 1 es que consideran una mala ejecución y 5 significa una ejecución correcta.	43
4.6. Respuestas de los usuarios donde indican si los árboles de comportamiento generados sirven como punto de partida (5) o no (1)	44
4.7. Respuestas de los usuarios sobre la facilidad para modificar los árboles de comportamiento generados, donde 1 significa que es difícil y 5 significa que es fácil.	45
4.8. Respuestas de los usuarios sobre su opinión de si la herramienta puede disminuir (5) o no (1) la carga de trabajo de los diseñadores.	45

Introducción

Los videojuegos están compuestos por múltiples elementos que interactúan entre sí para construir una experiencia coherente e inmersiva para el jugador, entre los que se encuentran el avatar del jugador, los escenarios, los objetos interactivos y, de forma habitual, los personajes no jugables. Estos personajes no jugables (NPC, por sus siglas en inglés de Non-Player Character) desempeñan un papel relevante en la construcción de mundos virtuales, ya que son responsables de proporcionar interacciones, responder a las acciones del jugador y desarrollar comportamientos coherentes con el contexto del juego.

El desarrollo de estos comportamientos constituye una tarea que involucra tanto aspectos de diseño como de implementación. Habitualmente, los diseñadores determinan las acciones, rutinas e interacciones que debe desarrollar un NPC, mientras que los programadores se encargan de trasladar estas especificaciones a una implementación ejecutable. Para ello pueden emplearse diferentes técnicas de inteligencia artificial, entre las que destacan las máquinas de estados finitos (Finite State Machines, FSM) y los árboles de comportamiento (Behavior Trees, BT), siendo estos últimos el principal objeto de estudio de este trabajo.

El desarrollo de comportamientos para NPCs es, además, un proceso inherentemente iterativo. Durante la implementación es necesario comprobar el comportamiento obtenido, detectar situaciones no previstas y modificar la lógica para adaptarla a los requisitos del juego. Como consecuencia, diseñadores y programadores pueden verse obligados a repetir sucesivamente las fases de diseño, implementación, evaluación y modificación de los comportamientos. En el caso de los árboles de comportamiento, estos cambios implican con frecuencia crear, reorganizar o modificar nodos, condiciones y relaciones de ejecución, lo que puede incrementar el esfuerzo necesario a medida que aumenta la complejidad del comportamiento.

A este coste de desarrollo se añade la necesidad de disponer de conocimientos técnicos específicos para construir y modificar correctamente estas estructuras. La utilización de un sistema basado en árboles de comportamiento requiere comprender tanto su lógica de ejecución como los elementos que lo componen y sus relaciones, lo que puede dificultar su utilización por parte de desarrolladores con menor experiencia en inteligencia artificial o por perfiles no especializados en programación.

En este contexto, la generación automática de árboles de comportamiento a par-

tir de descripciones realizadas en lenguaje natural constituye una posible vía para reducir el esfuerzo asociado a su creación y modificación. Permitir que un usuario pueda describir el comportamiento deseado mediante una especificación en lenguaje natural y obtener de forma automática una estructura ejecutable podría reducir parte del trabajo manual necesario durante el desarrollo, facilitar la iteración sobre los comportamientos y disminuir las barreras técnicas asociadas a su implementación.

La aparición de los modelos grandes de lenguaje (Large Language Models, LLMs), capaces de interpretar instrucciones en lenguaje natural y generar estructuras complejas a partir de ellas, abre una nueva posibilidad para automatizar este proceso. Por este motivo, este trabajo estudia la utilización de modelos de lenguaje para la generación automática de árboles de comportamiento destinados al control de personajes no jugables, analizando las posibilidades y limitaciones de este enfoque en el contexto del desarrollo de videojuegos.

1.1. Motivación

La creciente complejidad de los videojuegos ha incrementado también la complejidad de los comportamientos que deben implementar sus personajes no jugables. A medida que se incorporan un mayor número de acciones, situaciones y posibilidades de interacción, las estructuras encargadas de controlar estos comportamientos requieren un mayor número de elementos y relaciones entre ellos. En este contexto, los árboles de comportamiento presentan características como la modularidad y la escalabilidad que resultan especialmente adecuadas para representar comportamientos complejos ([Lim et al. \(2010\)](#)).

Sin embargo, estas características no eliminan el coste asociado a la creación y mantenimiento de los árboles. La construcción manual de comportamientos puede resultar laboriosa y requerir conocimientos especializados, especialmente cuando los comportamientos deben modificarse o ampliarse de manera iterativa. La literatura señala precisamente el esfuerzo que supone para los diseñadores desarrollar comportamientos de NPC y proponen mecanismos destinados a automatizar su expansión ([Florez-Puga et al. \(2008\)](#)). Asimismo, hay artículos que ponen de manifiesto la necesidad de facilitar la interacción entre los aspectos de diseño y los aspectos técnicos de la implementación de comportamientos ([Llansó et al. \(2009\)](#)).

La dificultad de autoría de los árboles de comportamiento también ha motivado la investigación de herramientas destinadas a facilitar su creación. Hay autores que proponen una herramienta de autoría basada en una interfaz visual que busca simplificar la construcción de estos árboles y acercar su edición al proceso de diseño ([Becroft et al. \(2011\)](#)). Estos trabajos muestran que la reducción del esfuerzo necesario para especificar y modificar comportamientos constituye una problemática anterior a la aparición de los LLMs.

Como consecuencia, se han desarrollado diferentes aproximaciones para automatizar total o parcialmente la construcción de los árboles de comportamiento. Entre ellas se encuentran métodos basados en la reutilización de comportamientos, técnicas de evolución automática y métodos de síntesis a partir de especificaciones formales, demostrando que la generación de árboles de comportamiento constituye una

línea de investigación consolidada. Estas aproximaciones permiten reducir parte del trabajo manual, aunque generalmente requieren representaciones estructuradas del problema, funciones de evaluación, demostraciones o conocimientos específicos del dominio.

La aparición de los LLMs introduce una nueva posibilidad para abordar este problema, ya que estos modelos permiten procesar instrucciones expresadas en lenguaje natural y generar estructuras a partir de ellas. En lugar de requerir que el usuario defina directamente la estructura interna del árboles de comportamiento, puede plantearse que describa el comportamiento deseado mediante lenguaje natural y que el sistema se encargue de realizar la traducción a una representación estructurada. Esta aproximación permite explorar una interacción más cercana al proceso de diseño y puede reducir las barreras técnicas asociadas a la autoría de comportamientos.

En este contexto, este trabajo se sitúa en la intersección entre la investigación sobre generación automática de los árboles de comportamiento y los avances recientes en LLMs.

1.2. Objetivos

El objetivo general de este proyecto es la implementación de una herramienta que puedan usar de forma sencilla e intuitiva diseñadores, tanto expertos como los que no tienen una base técnica, para poder generar árboles de comportamiento de una manera más eficaz que mediante la creación manual y con una calidad similar a la humana, con el fin de aliviar su trabajo. No se busca sustituir el trabajo de un diseñador, sino aportarles una herramienta que les devuelva unas estructuras fácilmente modificables para matizar ciertos aspectos o cambiar de forma sencilla los comportamientos generados. Para cumplir estos objetivos se han propuesto las siguientes metas durante el trabajo:

- Implementación de una pila de tareas y condicionales necesarias para generar diferentes comportamientos.
- Diseño e implementación de diferentes escenarios en los que se pueda probar en tiempo real y de forma visual los comportamientos buscados.
- Comparar diferentes métodos de implementación y modelos de lenguaje (tiempos de ejecución y calidad de las generaciones) para elegir el idóneo para cumplir el objetivo general.
- Implementación de la herramienta generadora usando diferentes técnicas de IA que consigan una generación óptima.
- Implementación de un sistema de validación de los árboles generados para comprobar que son válidos en cuanto a respeto de dependencias de nodos y cumplimiento de objetivos.
- Realizar un plan de pruebas para validar el cumplimiento de los objetivos generales.

1.3. Plan de trabajo

El plan de trabajo consiste en varios pasos:

1. **Noviembre - Febrero:** Desarrollo e implementación de varios escenarios en un motor de videojuegos que contengan lo necesario para la generación: una pila de tareas y condicionales, capacidad para ejecutar los BTs generados y una forma de comunicarse con la herramienta de generación.
2. **Febrero - Abril:** Implementación de una herramienta que, mediante grandes modelos de lenguaje, sea capaz de generar BTs a través del lenguaje natural capaces de satisfacer un comportamiento buscado.
3. **Abril - Junio:** Desarrollo de un sistema de validación automática de los BTs generados.
4. **Julio - Agosto:** Diseño y posterior ejecución de pruebas con usuarios objetivo para medir el correcto cumplimiento de los objetivos.

1.4. Metodología y tecnología

Para desarrollar este proyecto se han usado diferentes herramientas comúnmente utilizadas en el desarrollo de videojuegos con el objetivo público en mente.

- **Unreal Engine 5.7:** Unreal Engine es uno de los motores líderes de la industria, caracterizado por su robustez técnica. Además incorpora un sistema nativo de árboles de comportamiento dirigidos por eventos y un componente de Blackboard, necesario para la interacción del personaje no jugable con el entorno. Este motor ha sido seleccionado por su amplio uso en la industria, facilitando que los diseñadores hacia los que está enfocada la herramienta se encuentren en un entorno conocido. En esta herramienta se ha creado una pila de tareas y condicionales necesarias para la creación de comportamientos, así como varios escenarios para probar las generaciones de los árboles de comportamiento.
- **Python 3:** Este es un lenguaje de programación de alto nivel ampliamente extendido debido a su versatilidad y posición como uso común en el desarrollo de inteligencia artificial. Al ser el backend del proyecto, y siguiendo con los objetivos del uso de esta herramienta, la sección desarrollada con este lenguaje es completamente opaca al usuario. La herramienta desarrollada con Python es quien se encarga de toda la generación de los árboles, gracias a módulos disponibles de orquestación de LLMs, siendo Langchain el que ha sido usado en este trabajo.
- **Hugging Face:** Es un ecosistema de código abierto para la distribución de modelos de Machine Learning, librerías y datasets del Procesamiento de Lenguaje Natural. En este trabajo ha sido usado para crear un sistema RAG, siendo el

origen del repositorio de donde se han sacado los datos y quien provee el modelo de embeddings.

- **Mistral AI:** Es una compañía de modelos de inteligencia artificial generativa que ofrece modelos grandes de lenguaje. Uno de los modelos usados a lo largo del trabajo ha sido Mistral Large 3 a través de la API de esta empresa. Este modelo cuenta con 675B de parámetros totales y 41B activos, y está diseñado como un sistema de “Mezcla de Expertos”, haciéndolo un modelo robusto que puede cumplir con el propósito de la herramienta.
- **Ollama:** Se trata de una herramienta de código abierto diseñada para gestionar y ejecutar modelos de lenguaje de manera local, garantizando una baja latencia y la ausencia de costes por llamadas de APIs. Este entorno hospeda y ejecuta localmente otro de los modelos usados en el trabajo, el modelo llama3:8B, el cual cuenta con 8B de parámetros. Al ser un modelo más ligero no es tan preciso como el mencionado previamente, pero su uso ha resultado ser efectivo en este trabajo.

1.5. Estructura del documento

Este documento ha sido estructurado de acuerdo con el plan de trabajo mencionado previamente:

1. **Capítulo 2: Estado de la cuestión.** En este capítulo se presenta el marco teórico y tecnológico en el que se encuentra este trabajo. En este capítulo se repasan diferentes métodos de generación de comportamientos usados en videojuegos, tanto su definición como su evolución en la industria y academia, remarcando el interés especial en los árboles de comportamiento. También se cuenta la evolución y el contexto actual de los modelos de lenguaje, para, finalmente, contextualizar sobre la línea de investigación del proyecto de la generación automática de comportamientos.
2. **Capítulo 3: Herramienta de generación de árboles de comportamiento.** En este capítulo se describe el desarrollo de la herramienta implementada en este trabajo. Este capítulo se divide en cuatro secciones:
 - a) **Diseño del pipeline del modelo propuesto para la generación de árboles de comportamiento:** Se describe el diseño del flujo de la herramienta ideado para satisfacer los objetivos propuestos.
 - b) **Descripción del entorno en un motor de videojuegos:** Se detalla el entorno creado en un motor de videojuegos para la implementación de varias tareas y condicionales necesarios para generar diferentes tipos de comportamiento, así como el desarrollo de varios escenarios donde se pueden probar los árboles de comportamiento.
 - c) **Herramienta de generación de árboles de comportamiento:** Se explica la implementación de la herramienta que genera un archivo JSON

con un árbol de comportamiento desde la descripción del comportamiento buscado a través del lenguaje natural.

- d) **Validación automática de los árboles de comportamiento:** Se describen los métodos diseñados para la validación de los árboles de comportamiento: la validación del cumplimiento de dependencias entre tareas y del cumplimiento de un objetivo. Asimismo se describen varias pruebas que se han hecho para comprobar la validez del sistema.
- 3. **Capítulo 4: Evaluación y resultados.** En este capítulo se relatan las pruebas de evaluación con usuarios diseñadas y ejecutadas para validar una serie de hipótesis, así como se repasan los resultados dados.
- 4. **Capítulo 5: Discusión.** En este capítulo se discute la implementación y los resultados dados por todas las secciones del proyecto, así como se comentan las diferentes limitaciones encontradas durante su desarrollo.
- 5. **Capítulo 6: Conclusiones.** En este capítulo se detallan las conclusiones que han salido de los resultados del proyecto, así como se plantea el trabajo a futuro dadas las limitaciones.

1.6. Enlace al repositorio

El código fuente de este trabajo se encuentra en un repositorio público de Github. La estructura del repositorio es de la siguiente manera:

- **Análisis:** Se encuentra el CSV de las respuestas de las pruebas de usuario y el *Jupyter Notebook* con el código para generar las gráficas (ver sección 4.2).
- **bt-dataset-parser:** Proyecto de Python en donde se descarga y se procesa el dataset necesario para mostrar el sistema RAG (ver sección 3.3).
- **BTGenerator:** Proyecto de Unreal que ha servido como base para los entornos (ver sección 3.2).
- **llm-bt-generator:** Proyecto de Python con la herramienta generadora de BTs (ver sección 3.3).
- **Memoria:** Código fuente en LaTeX de este documento.

El enlace al repositorio es el siguiente: <https://github.com/pablosm196/TFM>.

1.7. Uso de Inteligencia Artificial

Este documento ha sido escrito con el apoyo puntual de herramientas de Inteligencia Artificial generativa en tareas de redacción. Su único uso ha consistido en cimentar las ideas establecidas originalmente en el documento reformulando ciertos párrafos, que posteriormente han sido revisados y editados. En todo caso la estructura del documento, así como las ideas que transmite y la responsabilidad final sobre el contenido recae en el autor.

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta el estado del arte de los distintos elementos que componen el trabajo. Estos campos son los comportamientos de los personajes no jugador (NPCs) en videojuegos, los modelos de lenguaje y la generación automática de comportamientos.

2.1. Comportamientos de NPCs en videojuegos

Como se ha mencionado en la sección 1, a medida que los videojuegos han ido creciendo y evolucionando se han tenido que utilizar diferentes técnicas para modelar comportamientos cada vez más grandes y complejos (Gajardo et al. (2019)).

2.1.1. Máquinas de estado finitas

Las máquinas de estados finitas (FSM según las siglas en inglés de Finite State Machine) se definen como un conjunto de estados s_i y un conjunto de transiciones s_i, s_j que están definidas por una relación de tipo acción/condición, donde la condición actúa como el evento desencadenante para realizar la transición y la acción representa la tarea específica que se ejecuta al cambiar de estado (Ben-Ari y Mondada (2018)). Según su definición se puede ver que resulta muy útil para modelar el comportamiento en los videojuegos (se puede ver un ejemplo en la figura 2.1), ya que se pueden ver a los NPCs como agentes que realizan acciones (permanecen en estados) constantemente, siendo las transiciones la terminación de las acciones (cuando pasan a un estado siguiente) o un factor externo.

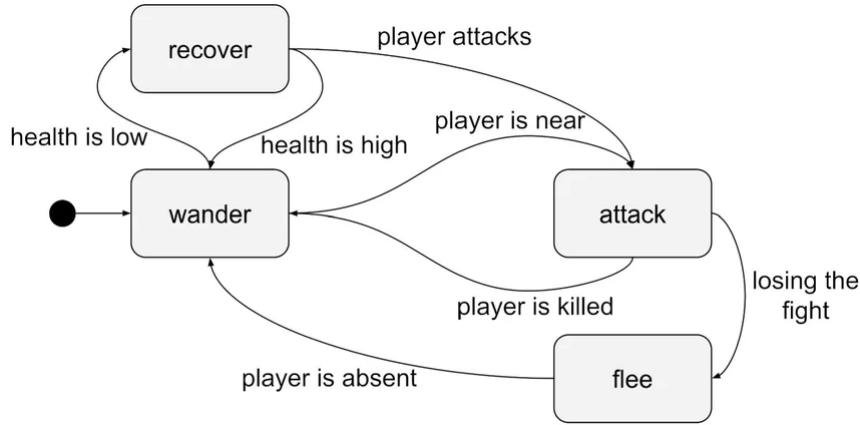


Figura 2.1: Ejemplo de un comportamiento típico de un NPC mediante una FSM (Uludagli y Oguz (2023))

Esto se puede ver en algunos trabajos donde los autores presentan un videojuego en el que implementan las FSM en diferentes NPCs para que muestren un comportamiento más consistente, realista y reactivo por su simplicidad a la hora de desarrollar, dando como resultado un comportamiento inteligente y autónomo (Fathoni et al. (2020)). Otro ejemplo del uso histórico de las FSMs en videojuegos se puede ver en artículos donde los autores implementan una FSM con probabilidad de transiciones distinta a 1 para emular de forma más realista el comportamiento de caza de los lobos, logrando así una experiencia más realista en inmersiva para el usuario (Lee (2014)). Sin embargo, las FSM no están tan extendidas actualmente, ya que la literatura afirma que las FSM tienen un problema de reutilización, flexibilidad y escalabilidad con comportamientos complejos (Hu et al. (2011)). Aún así actualmente se siguen utilizando, como se puede ver en trabajos recientes donde implementan FSM de forma clásica para NPCs o con combinaciones de FSM con técnicas de Machine Learning como Q-Learning y DQN para controlar NPCs de un RPG (Ganapathi et al. (2024) y KimDongju y KohSeok-Joo (2025)).

2.1.2. Máquinas de estado finitas jerárquicas

Las máquinas de estado finitas jerárquicas (HFSM por las siglas en inglés de Hierarchical Finite State Machines) se definen como una extensión de una FSM cuyos estados puedan ser otras FSM contenidas, llamados “superestados” (Yannakakis (2001)). Aunque todas las HFSM se pueden convertir en FSM mediante un proceso de *flattening* reemplazando de forma recursiva los superestados por sus máquinas internas, las HFSM han solucionado problemas de reutilización de componentes y simplicidad descriptiva. Esta reutilización ha permitido que se extendiese su uso en videojuegos con un rendimiento ligeramente mejor o similar a las FSM (Hu et al. (2011) y Fauzi et al. (2019)). En la figura 2.2 se muestra un ejemplo de una HFSM en videojuegos.

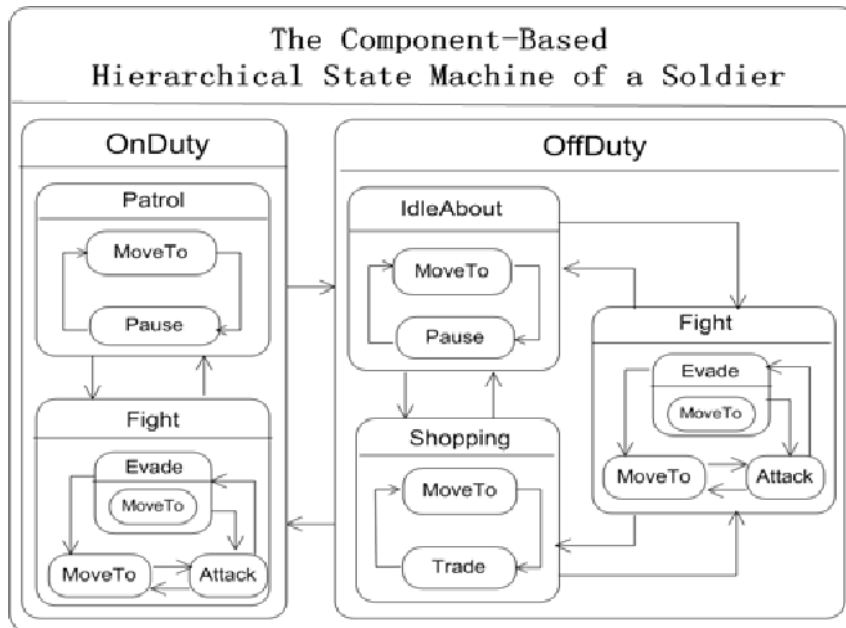


Figura 2.2: Ejemplo de un comportamiento de un NPC modelado en una HFSM (Hu et al. (2011)).

De forma comercial también se han usado las HFSM (Thompson (2018)). Otra forma de uso de las HFSM en videojuegos no es para comportamientos de NPCs, sino para simplificar el sistema de animaciones de videojuegos (Nur Fitriani Dewi et al. (2023)). El motor de videojuegos Unity Engine también estructura su sistema de animaciones¹ en forma de HFSM para la simplicidad del usuario, como se puede apreciar en el estado de color naranja (estado “Idle”) de la figura 2.3.

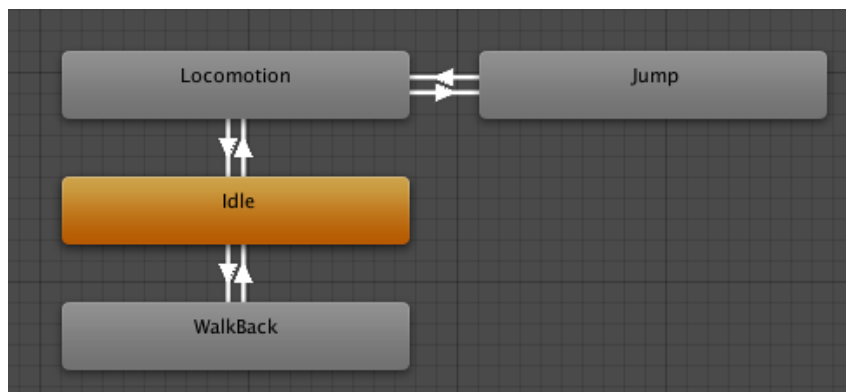


Figura 2.3: Captura del sistema de animaciones de Unity, estructurado en forma de una HFSM en la que los nodos naranja son “superestados”. Fuente: <https://docs.unity3d.com/560/Documentation/Manual/StateMachineBasics.html>

A pesar de aliviar parcialmente el problema de la duplicidad de código las HFSM siguen manteniendo problemas de escalabilidad y reutilización en videojuegos, además de que siguen siendo difícil de escalar (Sekhavat (2017) y Iovino et al. (2025)).

¹Enlace a la documentación de Unity: <https://docs.unity3d.com/6000.7/Documentation/Manual/AnimationStateMachines.html>

2.1.3. Árboles de comportamiento

Los árboles de comportamiento (conocidos como BTs) se definen como árboles dirigidos formados por dos tipos de nodos: nodos hoja y nodos de composición en las que las transiciones no son unidireccionales (a diferencia de las FSM y las HFSM), sino llamadas de función bidireccionales en las que el nodo padre envía una señal periódica de activación y el hijo le responde con tres posibles estados: “Success”, “Failure” o “Running” (Colledanchise y Ogren (2016)). Un ejemplo de BT lo podemos encontrar en la figura 2.4.

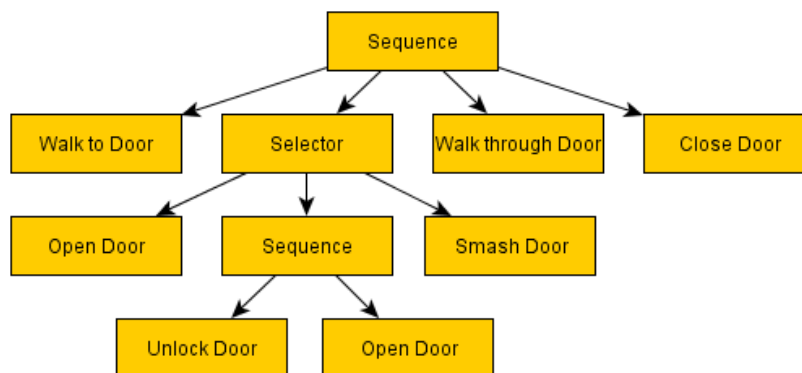


Figura 2.4: Ejemplo de un árbol de comportamiento para un NPC que debe pasar por una puerta. Fuente: <https://outforafight.wordpress.com/2014/07/15/behaviour-behavior-trees-for-ai-dudes-part-1/>

Los BTs surgieron en el mundo de la programación de inteligencia artificial de los videojuegos (estableciendo el videojuego “Halo 2” como hito histórico) como forma de solucionar los problemas de escalabilidad, acoplamiento, adaptabilidad y reutilización que presentaban las FSM (Iovino et al. (2022)). Además, la literatura comenta que este modelo resultó ser muy valioso también para la robótica (Iovino et al. (2020a) y Ögren y Sprague (2022)). Actualmente hay bastante bibliografía sobre BTs, sus implementaciones y posibles mejoras. Por ejemplo, hay trabajos donde nos hablan de BTs que no son evaluados cada tick sino cuando sucede un evento (llamados Event-Driven Behavior Trees o EDBT), mejorando así la cooperación en entornos multi-agente (Agis et al. (2020)). También se han hecho varias ampliaciones a los BTs en videojuegos, como los EDBT, los árboles de comportamiento emocionales (EmBT) que alteran dinámicamente la prioridad de los nodos hijos dependiendo de pesos emocionales (aportando así más humanidad a los NPCs) y la evolución de los comportamientos (Evolving Behavior), BTs que utilizan la programación genética para automatizar su optimización (Guo y Hou (2025)).

Publicaciones científicas hablan del uso de los BTs como herramienta estándar de la industria (sin dejar de usar otros métodos como las ya mencionadas FSM), aunque no existe un consenso sobre la descripción de sus propiedades ni sobre su semántica (Tremblay et al. (2026) y Gugliermo et al. (2024)). Es por ello que este trabajo se va a centrar en la implementación de BTs específica de Unreal Engine.

2.1.3.1. Árboles de comportamiento en Unreal Engine

Unreal Engine se ha convertido en uno de los principales motores de la industria de los videojuegos, así como en el ámbito de la investigación con BTs (Partlan et al. (2022)). Los BTs de Unreal Engine están basados en EDBT, como se menciona en la documentación oficial de Unreal Engine², incrementando de esta manera el rendimiento del juego. Esto se hace gracias al uso de la Blackboard, un componente de memoria que almacena datos y variables de entrada/salida que usa el BT para ciertas tomas de decisiones de ejecución.

Los tipos de nodos que proporciona Unreal se dividen en dos tipos: las composiciones y las tareas (a excepción de otros tipos de BTs, que también consideran los condicionales como nodos adicionales). Los nodos de tipo tarea son los nodos hoja del árbol y los que aportan la lógica del NPC. Cuando se evalúan estos nodos pueden devolver el estado de “Success”, “Failure”, “In Progress” o “Aborted”.

Los tipos compuestos son nodos que controlan el flujo de ejecución del BT:

- **Selector:** Ejecuta el primer hijo que cumpla con ciertas condiciones.
- **Sequence:** Ejecuta todos sus hijos en orden o hasta que uno devuelva “Failure”.
- **Simple Parallel:** Tiene dos conexiones: la tarea principal y la rama secundaria. Ejecuta las dos a la vez y se puede configurar para que su ejecución acabe con la tarea principal o tenga que esperar a la rama secundaria.

Como se ha mencionado anteriormente los condicionales, llamados “decoradores” en Unreal Engine, no son nodos, sino que se unen a los nodos para comprobar si pueden ser ejecutados o no. A los nodos también se le pueden unir “Servicios”, los cuales se ejecutan cada cierto tiempo determinado mientras su rama esté siendo ejecutada.

En la figura 2.5 se puede ver un ejemplo de un BT realizado en Unreal Engine.

²Enlace a la documentación oficial de Unreal: <https://dev.epicgames.com/documentation/unreal-engine/behavior-tree-in-unreal-engine---overview?lang=en-US>

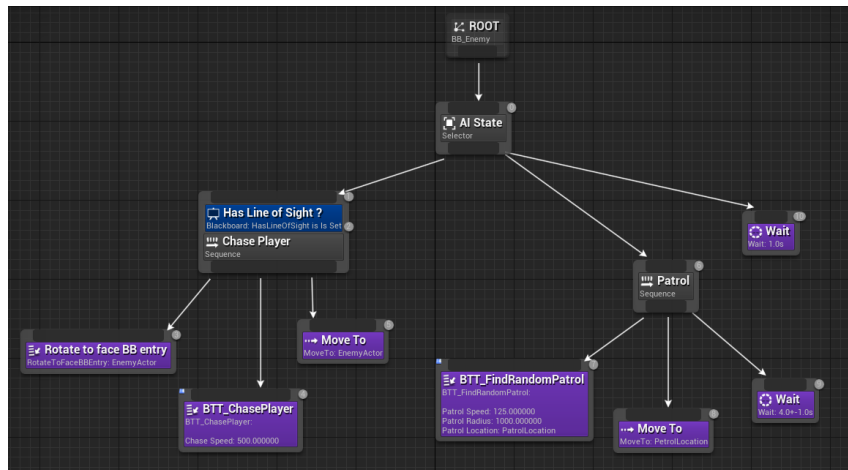


Figura 2.5: Ejemplo de un árbol de comportamiento de Unreal Engine de un NPC que patrulla y persigue al jugador. Fuente: <https://dev.epicgames.com/documentation/unreal-engine/behavior-tree-in-unreal-engine---overview>

2.2. Modelos de lenguaje

Los modelos de lenguaje son sistemas de IA diseñados para comprender, procesar y generar texto. Al principio se basaban en modelos de lenguaje estadístico, de forma que gracias al análisis de corpus de textos y el uso de n-gramas (una instancia específica de un modelo de Markov) se conseguían estimar las probabilidades de una palabra gracias a un contexto limitado (Jelinek y Mercer (1980)). Aunque posteriormente surgieron trabajos donde usaban el concepto de máxima entropía para que los modelos sean ligeramente sensibles al contexto que mejoraban estos modelos probabilísticos, seguían estando muy limitados por su baja capacidad de contexto y nulo razonamiento detrás de las respuestas (Berger et al. (1996)).

En trabajos posteriores se introducen modelos de lenguaje probabilístico neuronal con el fin de aprender una representación distribuida para cada palabra del vocabulario asociándola con un vector continuo de características reales y una función de probabilidad condicional de la siguiente palabra dependiendo de su contexto, consiguiendo una reducción de la perplejidad (métrica de los modelos de lenguaje que mide la precisión de predicción de la siguiente palabra) de entre el 10% y el 20% y permitiendo aprovechar contextos de entrada más largos y más distribuidos, no solo teniendo en cuenta las N palabras anteriores (Bengio et al. (2003)). Este trabajo permitió una evolución de modelos de lenguaje probabilísticos a los basados en redes neuronales, como las Redes Neuronales Recurrentes (RNNs, Elman (1990)) o las Long-Short Term Memory (LSTM, Hochreiter y Schmidhuber (1997)). Nos podemos encontrar trabajos donde se introduce el concepto de las RNN Encoder-Decoder para su uso en traducción de textos, mejorando el flujo de memoria de un RNN convencional y consiguiendo un rendimiento significativamente superior gracias a la preservación automática de estructuras semánticas y sintácticas (Cho et al. (2014)). A pesar de la mejora que aplicaron las redes neuronales previamente

mencionadas frente a los modelos puramente estadísticos, todavía tenían imperfecciones debido al desvanecimiento de contextos en textos amplios o al amplio coste de procesamiento debido a su secuencialidad. Aún con estas limitaciones los LSTM fueron fundamentales en otro hito de los modelos de lenguaje con la arquitectura ELMo (Embeddings for Language Models), la cual gracias a la suma ponderada de los resultados de las capas LSTM bidireccionales es capaz de generar una representación vectorial de las palabras (embeddings) contextualizada a nivel semántico y sintáctico, a diferencia de las estructuras estáticas que se utilizaban previamente como Word2Vec (Peters et al. (2018) y Mikolov et al. (2013)).

Más adelante se presentó el modelo Transformer, que logra suplir los problemas mencionados de las redes recurrentes (Vaswani et al. (2023)). Esto lo consiguen mediante una arquitectura Encoder-Decoder que contiene unos mecanismos de atención multipropósito y una codificación posicional que inyecta información sobre el orden secuencial, prescindiendo así de los mecanismos de recurrencia. Se puede ver un diagrama de la arquitectura en la figura 2.6 Los mecanismos de atención logran relacionar cualquier pareja de tokens de una misma secuencia sin importar la distancia física que les separe, permitiendo aprender dependencias a largo alcance y la paralelización del entrenamiento.

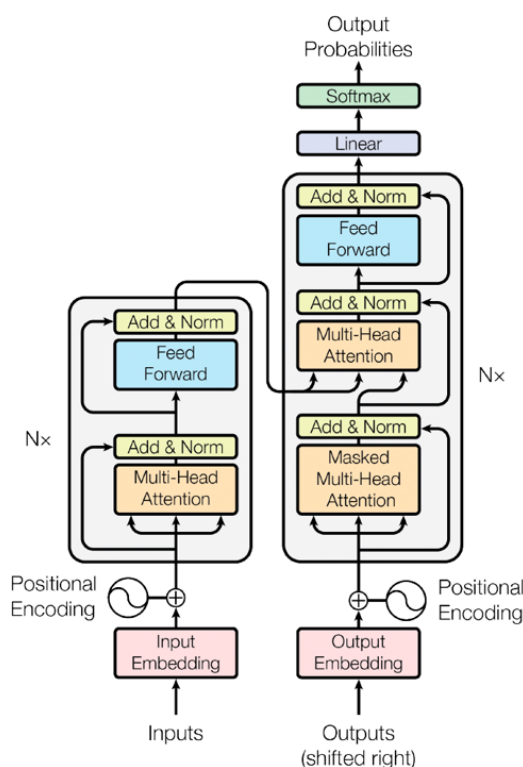


Figura 2.6: Diagrama de las distintas capas de la arquitectura Transformer. Fuente: <https://aws.amazon.com/es/what-is/transformers-in-artificial-intelligence/>

Gracias al surgimiento de los Transformers empezaron a aparecer otras arquitecturas que mejoraban las tareas del procesamiento del lenguaje natural (NLP por

las siglas en inglés de Natural Language Processing), en donde podemos destacar la arquitectura BERT (Devlin et al. (2019)). Esta arquitectura utiliza un codificador Transformer bidireccional multipropósito que para el entrenamiento enmascara de forma aleatoria una serie de tokens (Masked Language Model o MLM) para la predicción del siguiente token según su contexto anterior y posterior, logrando así una bidireccionalidad profunda y logrando un mayor desempeño en tareas como reconocimiento de entidades. Para ajustar el modelo a una tarea específica se realiza un fine-tuning (reentrenamiento de ciertas partes del modelo sobre los pesos establecidos) en el que se cargan los parámetros preentrenados y se vuelven a entrenar todos de forma conjunta (end-to-end) con datos etiquetados de la tarea que se busca. Esto difiere y mejora otros modelos de representación del lenguaje como ELMo, cuya estrategia de transferencia se basa en la extracción de características (Feature-Based), donde las representaciones contextuales preentrenadas se integran de forma estática como características adicionales dentro de arquitecturas secundarias.

La idea del entrenamiento previo y posterior Fine-tuning surge en otros artículos en donde además introducen el modelo GPT-1 (Radford y Narasimhan (2018)). Este modelo utiliza la arquitectura decodificadora de un Transformer multicapa, el cual en el entrenamiento aplica operaciones de autoatención enmascarada sobre los tokens de contexto para predecir la salida para posteriormente aplicarle Fine-tuning para tareas específicas, consiguiendo una mejora respecto a modelos anteriores en tareas como razonamiento del sentido común o comprensión lectora.

Como evolución del modelo anterior surge GPT-3, cuya arquitectura está basada también un Transformer decoder de 175B de parámetros con la novedad de la introducción de patrones de atención localmente y densamente agrupada de forma alternada (Brown et al. (2020)). En el artículo donde lo presentan también surge el aprendizaje en contexto (In-Context Learning), el cual se basa en conseguir que un modelo de lenguaje consiga realizar una tarea sin necesidad de cambiar los pesos del modelo mediante Fine-tuning, consiguiendo superar la necesidad de grandes cantidades de datos etiquetados. Esto se puede realizar mediante la descripción de la tarea en lenguaje natural y unos pocos ejemplos (few-shot), un ejemplo (one-shot) o ningún ejemplo (zero-shot). Con solo este tipo de aprendizaje el modelo fue capaz de mejorar el rendimiento de varias tareas de NLP como traducción o completado de texto así como de conseguir capacidades emergentes como la aritmética o la generación de noticias sintéticas.

También surge el concepto de Instruction Tuning, el cual mejoran las capacidades de aprendizaje en entornos zero-shot (Wei et al. (2022)). Lo consiguen mediante el uso de Fine-tuning sobre una colección de tareas descritas mediante instrucciones en lenguaje natural, presentando el modelo FLAN basado en un Transformer decoder de 137B parámetros. De esta forma consiguieron superar en gran medida al modelo GPT-3 en entornos zero-shot y en menor medida en entornos few-shot.

Junto a estas arquitecturas empezaron a surgir aplicaciones que ofrecían el uso de LLMs en formato chatbot, como OpenAI³ con ChatGPT o Anthropic⁴ con Claude así como acceso a sus APIs, permitiendo a usuarios no expertos el uso de LLMs para fines generalistas. Adicionalmente surgieron frameworks de orquestación de

³Página oficial de OpenAI: <https://openai.com/es-ES/>

⁴Página oficial de Anthropic: <https://www.anthropic.com/>

modelos como Langchain⁵ o herramientas que permiten la ejecución de modelos en una máquina local, como es el caso de Ollama⁶

Gracias al surgimiento del In-Context Learning y del Instruction Tuning se empezaron a poder pedir tareas específicas a los LLMs mediante los prompts, aunque se dan casos donde los modelos no consigan una buena precisión. Debido a esto se investigaron diferentes métodos de alinear el razonamiento de los modelos a través de refinar el prompt, dando lugar a un conjunto de técnicas que se conocen como *prompt engineering*. Algunas de estas técnicas ya han sido mencionadas anteriormente, como el zero-shot para casos de Instruction Tuning o few-shot/one-shot para In-Context Learning. Nos podemos encontrar con otras técnicas como Chain-of-Thought, la cual consiste en obligar al modelo a realizar pasos intermedios de razonamiento internamente para mejorar el rendimiento de las tareas complejas (Wei et al. (2023)). Otra técnica útil para tareas complejas es Least-To-Most Prompting, presentada en el artículo Zhou et al. (2023) y que consiste en descomponer la tarea general en subtareas sucesivas para ir resolviéndolos de manera incremental. En revisiones sistemáticas nos podemos encontrar con una lista de varias técnicas más de *prompt engineering* (Schulhoff et al. (2025)).

Una vez estudiadas las capacidades de razonamiento de los LLMs, se abrió un camino para el estudio de agentes basados en ellos: sistemas autónomos con modelos de lenguaje integrados que son capaces de percibir, planificar e interactuar con su entorno para completar tareas complejas. En el paradigma ReAct nos encontramos un LLM que genera alternadamente trazas de razonamiento en lenguaje natural que ayuda al razonamiento para actuar y acciones específicas para la tarea, permitiendo interactuar con el entorno y actuar para razonar (Yao et al. (2023)). Otro avance hacia los agentes LLMs nos lo encontramos en otros trabajos donde se pueden ver como un agente es capaz de interactuar con APIs de forma correcta que le permiten realizar mejor tareas como operaciones aritméticas, además de demostrar que se le puede dar una cierta autonomía de interacción con el entorno sin intervención humana a los LLMs (Schick et al. (2023)). En algunas revisiones sistemáticas nos hablan sobre el uso de herramientas (tool calling) en trabajos recientes, organizándolo en cuatro etapas: planificación de la tarea, selección de la herramienta, llamada a la herramienta y respuesta final (rong WEN (2025)). En cuanto a la interacción y planificación de los agentes, hay autores que estudian como un LLM puede utilizar feedback del entorno para actualizar de forma interna sus planes, mientras que en otros argumentan que un LLM tiene dificultades en temas de planificación a largo plazo, por lo que proponen usar un planificador clásico entre medias cuando el problema está correctamente formalizado para suplir esta carencia (Huang et al. (2022) y Liu et al. (2023)). Finalmente, podemos encontrarnos con trabajos donde se introduce un sistema multiagente basado en agentes LLM sociales con una memoria y la capacidad de recuperar experiencias y de planificar, demostrando que el uso de este tipo de agentes consiguen unas interacciones sociales más creíbles que el uso de agentes clásicos (Park et al. (2023)). Nos podemos encontrar con revisiones sistemáticas sobre agentes autónomos LLM (Wang et al. (2024)).

Para que estos agentes puedan interactuar con herramientas y recursos externos,

⁵Página oficial de Langchain: <https://www.langchain.com/>

⁶Página oficial de Ollama: <https://ollama.com/>

se creó el protocolo MCP (Model Context Protocol) (Hou et al. (2026)). Éste es un estándar abierto cuyo objetivo es estandarizar la comunicación bidireccional entre los modelos LLM y las fuentes y herramientas externas mediante el desacople de la implementación de las herramientas de su uso. Para ello se hace el uso de la siguiente arquitectura:

- **MCP Host:** La aplicación que proporciona el entorno de ejecución.
- **MCP Client:** Componente dentro del Host que mantiene la relación con el servidor procesando solicitudes y respuestas.
- **MCP Server:** Proveedor que expone las herramientas, los recursos y una serie de prompts para plantillas de tareas y flujos de trabajo preconfigurados.

En la figura 2.7 podemos ver un diagrama de la arquitectura MCP.

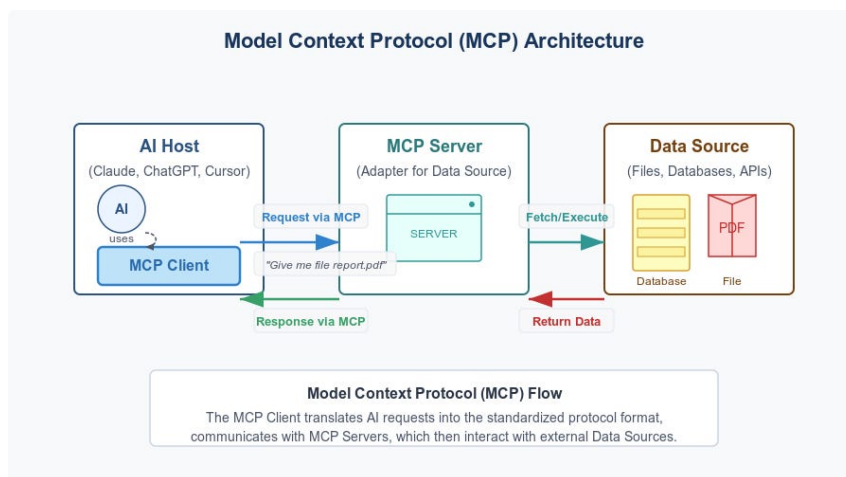


Figura 2.7: Diagrama de la arquitectura MCP. Fuente: <https://newsletter.diamant-ai.com/p/model-context-protocol-mcp-explained>

2.3. Generación automática de comportamientos

El problema de la generación de comportamientos de forma automática ya existía antes del surgimiento de los LLMs, cuando buscaban soluciones mediante algoritmos de IA clásica con, por ejemplo, autores que proponen recuperar y reutilizar comportamientos mediante CBR (siglas de Case-Based Reasoning), construyendo dinámicamente el comportamiento del NPC a través de una HFSM. (Florez-Puga et al. (2008)) Esta sección se va a centrar en los casos de generaciones automáticas de BTs.

2.3.1. Generación de árboles de comportamiento previos a LLMs

Desde el surgimiento de los BTs se ha buscado una forma de automatizar el proceso de su creación.

Por ejemplo, hay autores que utilizan diferentes técnicas de Machine Learning para generar de forma automática BTs ([Sagredo-Olivenza et al. \(2017\)](#)). Primero se utiliza el aprendizaje por demostración (LfD por las siglas en inglés de Learning from Demonstration) de forma que el diseñador realiza la tarea de la que se busca generar el BT, registrando las trazas del estado del entorno y las tareas que el NPC va ejecutando. Una vez se terminan de recoger esas trazas se generan diferentes modelos de Machine Learning que imitan el comportamiento del agente. Si este modelo es un árbol de decisión (usando el algoritmo de aprendizaje supervisado C4.5), éste se aplanar mediante una búsqueda en profundidad (DFS) para generar reglas lógicas y se unen mediante operadores AND y OR y, usando un algoritmo voraz, simplifica los nodos redundantes hasta construir el BT componiéndolo en un nodo de tipo “Parallel” o “Priority Selector”.

En el campo de la programación genética también se han hecho avances en este aspecto. Hay artículos en los que usan un videojuego comercial donde crean a mano un BT inicial cuyas operativas específicas se toman de forma aleatoria ([Lim et al. \(2010\)](#)). Es entonces cuando se aplican los algoritmos genéticos sobre esta base del árbol, donde el *crossover* se realiza mediante el intercambio de subramas enteras con el mismo objetivo para dar origen a nuevos comportamientos y la mutación se realiza en parámetros de nodos específicos o insertando nuevas ramas de comportamientos generadas al azar. La función de *fitness* se entrenó mediante métricas específicas del videojuego, como el potencial defensivo y atacante del NPC. Este sistema logró mejorar a la IA del videojuego en un 55 % de las partidas, demostrando que los BTs generados resultaron ser correctos. También existen ejemplos del uso de la programación genética para la generación de BTs en el ámbito de la robótica, donde además los autores mezclan la programación genética con aprendizaje por demostración ([Iovino et al. \(2020b\)](#) y [Iovino et al. \(2023\)](#)).

Finalmente nos encontramos con algoritmos de planificación para la generación de BTs ([Colledanchise et al. \(2019\)](#)). Para ello, los autores introducen una metodología para crear y modificar BTs en tiempo de ejecución mediante el algoritmo de planificación de retroencadenamiento (Backchaining) en el que se toman un conjunto de acciones con sus precondiciones y postcondiciones y las convierte en BTs atómicos bajo un nodo “Selector”. Una vez se procesan todas las acciones, se parte de un BT que contiene las condiciones finales deseadas y, cada vez que el árbol devuelva un estado “Failure” porque el objetivo no se cumple, se reemplaza esa condición por uno de los BTs atómicos que sea capaz de resolverla, repitiendo iterativamente este proceso hasta que se de un BT correcto. Si hay una violación de una precondición de otras partes ya predefinidas del árbol, el algoritmo detecta estas colisiones lógicas y las resuelve reordenando el BT desplazando el subárbol conflictivo para incrementar su prioridad.

2.3.2. Generación de árboles de comportamiento mediante LLMs en la robótica

Dado el gran avance en el campo de los LLMs se pueden encontrar varios trabajos que se sirven de éstos para la creación automática de BTs, especialmente en el campo de la robótica.

En uno de los primeros trabajos en este ámbito, los autores utilizan un modelo basado en Stanford Alpaca 7B al que realizan Fine-tuning mediante LoRA (modifican únicamente un grupo reducido de pesos de atención) con demostraciones generadas a través de otros modelos más grandes (text-davinci-003), consiguiendo generar BTs parecidos a los que podría crear un humano para comportamientos robóticos (Lykov y Tsetserukou (2023)). Otros trabajos siguen una metodología similar en la que utilizan modelos ligeros (de 7B de parámetros) en los que realizan Fine-tuning con datasets sintéticos (Izzo et al. (2024)). En otros artículos se estudian diferentes técnicas de In-Context Learning para generar BTs y, además, se comparan con modelos de menor tamaño a los que se les ha aplicado Fine-tuning y los evalúa en simulaciones, consiguiendo una mayor tasa de aciertos mediante el uso del In-Context Learning (Ao et al. (2025)).

Hay autores que también proponen el desarrollo y entrenamiento de un LLM especializado únicamente en la generación de BTs (Li et al. (2024)). Para ello proponen un pipeline que consta de los siguientes pasos:

1. Creación de un dataset mediante la generación de datos sintéticos usando búsquedas mediante Monte Carlo Tree Search (MCTS) combinadas con un sistema RAG.
2. Entrenamiento del modelo mediante aprendizaje auto-supervisado (MLM).
3. Fine-Tuning supervisado con datos de instrucciones de usuarios con su respectivo BT.
4. Creación de un framework (BTGen Agent) para la generación de los árboles que consta de un módulo de memoria (almacena contextos a corto y largo plazo), módulo de acción (ontología de acciones disponibles), módulo de planificación y módulo de perfil (configura al LLM para que adapte roles específicos).
5. Creación de una metodología de verificación y validación (V&V) de las capacidades y el rendimiento del LLM.

Finalmente, en otro ejemplo, hay autores que incorporan a su sistema de generación de BTs mediante ChatGPT un “intérprete de fallos”, el cual permite adaptar en tiempo de ejecución los árboles, consiguiendo una tasa de éxito del 89.6 % (Merino-Fidalgo et al. (2025)).

2.3.3. Generación de árboles de comportamiento mediante LLMs en videojuegos

A diferencia del campo de la robótica, la literatura sobre generación de BTs para videojuegos es escasa.

En un ejemplo, los autores nos presentan un framework en el que, al igual que en trabajos previos del campo de la robótica, se pueden generar BTs a través del lenguaje natural (Paduraru et al. (2025)) mediante la realización de un Fine-tuning al modelo “ollama3.1:8B Instruct” mediante LoRA gracias a un corpus mixto de árboles generados manualmente sobre dos videojuegos comerciales y árboles generados

mediante el modelo “GPT-4o”. Para generar los árboles se introducen las acciones disponibles en un almacén de vectores FAISS y se recuperan mediante etiquetado de roles semántico (SRL). Mediante pruebas con usuarios se vio que este agente generador alcanzó un 78 % de ejecución correcta en estructuras sencillas, y un 81 % de éxito en estructuras complejas gracias al post-procesamiento.

Para terminar, nos encontramos con autores que proponen un trabajo parecido mediante el modelo “GPT-5.2” siguiendo diferentes niveles de detallismo en el prompt del diseñador, donde va desde la descripción explícita de la estructura del BT buscado (en donde se consiguió un 100 % de precisión) hasta la delegación completa del diseño, pidiendo una descripción muy vaga del comportamiento buscado (ITO et al. (2026)).

Esta escasez de trabajos en el ámbito de los videojuegos frente al campo de la robótica hace ver que es conveniente la investigación de esta línea.

Herramienta de generación de árboles de comportamiento

Debido al proceso tedioso y costoso en tiempo de formación y producción que conlleva a los diseñadores la creación de árboles de comportamiento (BTs) de forma constante para idear a los diferentes personajes no jugador (NPCs), el objetivo general de este trabajo es la creación de una herramienta que sea capaz de generar BTs funcionales que sirvan de base para rebajar la carga de trabajo de los diseñadores. Para ello se busca un modelo que sea completamente opaco al usuario, que dé resultados en un tiempo inferior a la alternativa ya existente con una calidad similar y que sea sencilla de usar sin requerir muchos conocimientos técnicos para, además, ayudar a diseñadores con un perfil no tan técnico a desarrollar con mayor facilidad.

Con esta idea en mente y gracias a los avances que se han producido en el campo de los modelos del lenguaje, se ha desarrollado un modelo de herramienta basada en generación mediante LLMs, ya que ofrece la ventaja de que los usuarios puedan explicar el comportamiento deseado en lenguaje natural. Como esta herramienta debería ser de fácil uso para los diseñadores y programadores, se propone un modelo en el que, sin salir de un proyecto de Unreal Engine, se puedan generar los BTs mediante su propia interfaz.

3.1. Diseño del pipeline del modelo propuesto de generación de árboles de comportamiento

Antes de diseñar e implementar la herramienta se ha buscado tener claro el diseño de cuál debería ser el flujo de ejecución, así como los diferentes módulos que la iban a componer.

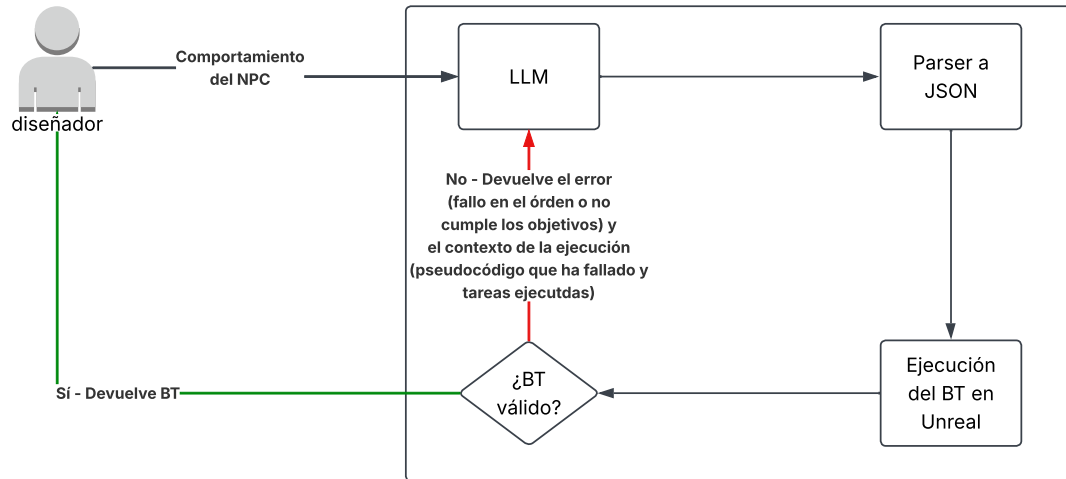


Figura 3.1: Diagrama donde se visualiza el flujo de la herramienta propuesta.

En la figura 3.1 se puede ver el flujo de ejecución propuesto:

1. El diseñador describe el comportamiento buscado para un NPC.
2. Fuera del entorno del motor de videojuegos se genera el árbol de comportamiento mediante LLMs en un formato JSON.
3. El motor de videojuegos traduce y crea el árbol de comportamiento generado.
4. Se ejecuta el árbol.
5. Opcionalmente se valida el árbol de forma automática. En caso de ser un árbol válido se termina la ejecución. En caso contrario se devuelve al modelo generador el árbol fallido y se le avisa del error.

Este pipeline busca la independencia entre el motor de videojuegos y los modelos LLM, así como una opacidad de la generación frente a los usuarios y la simplicidad para diseñadores que no tengan un perfil muy técnico.

En las siguientes secciones de este capítulo se explicará el entorno de pruebas desarrollado para este trabajo, así como la implementación de la generación de los BTs, la validación de éstos y las pruebas a usuarios realizadas.

3.2. Descripción del entorno en un motor de videojuegos

Lo primero que se ha necesitado es un entorno en el que se puedan implementar una serie de acciones y condicionales (controladores de flujos de ejecución) con el fin de tener la posibilidad de diseñar una serie de comportamientos para poder crear y ejecutar los árboles de comportamientos. Para ello, y por el enfoque de este

proyecto en el desarrollo de videojuegos, se han desarrollado una serie de escenarios en el motor de videojuegos Unreal Engine 5.7 debido también a su amplio uso en la industria y su robusto sistema de árboles de comportamiento¹. Son estos entornos los que se van a comunicar con el agente LLM de Python para, posteriormente, probar los árboles generados y comprobar su validez.

Estos entornos, de los que se pueden ver un ejemplo en la figura 3.2, constan de un agente (el NPC que va a seguir el comportamiento buscado) que contiene el componente de Behavior Tree necesario para la futura ejecución de un árbol de comportamiento. Además, contiene un componente de Blackboard, el cual es esencial porque constituye la memoria del NPC. Este consiste en una serie de variables entrada/salida de las que las tareas y condicionales pueden leer y/o escribir, siendo esta la manera que tienen los NPCs de interactuar con el entorno.

En la misma figura se puede apreciar la interfaz en la que el usuario debe introducir la información necesaria para la generación del árbol: la descripción del comportamiento buscado, el nombre del archivo con el que se va a guardar el árbol de comportamiento generado, las entradas de la Blackboard, el agente que va a ejecutar el árbol generado y, opcionalmente, el nombre del archivo de validación de la generación. Esta información la debe contener un Actor (los objetos del mundo de Unreal), llamado en la captura “BTGeneratorManager”, que tenga un componente llamado “BTGenerator”. Este componente de Actor es el encargado de lanzar el proceso de Python con la información que el usuario haya puesto y el que comienza la ejecución de las pruebas, siendo la clase principal de la herramienta en Unreal. Para comenzar la ejecución se debe pulsar el botón “GenerateBT” que se aprecia en la captura.

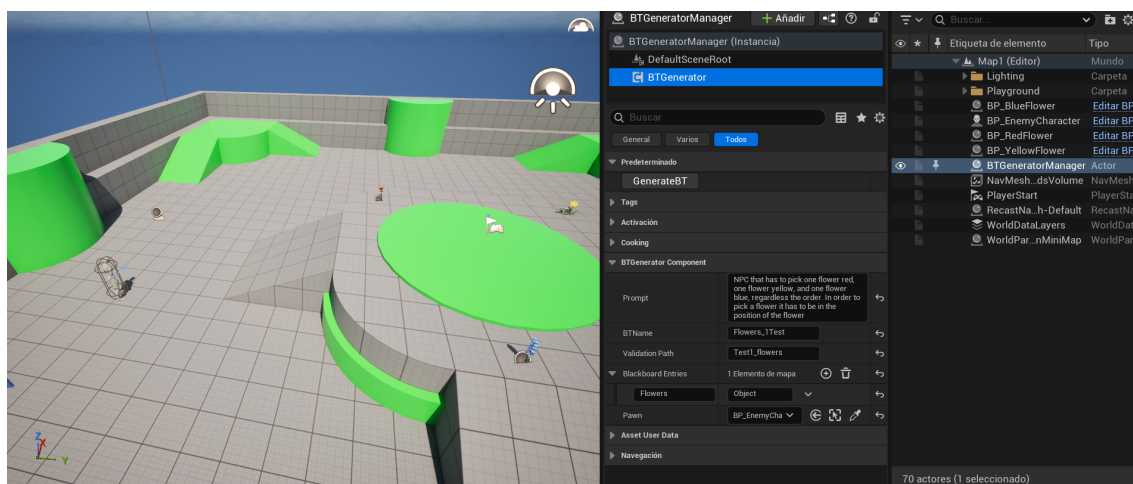


Figura 3.2: Captura de pantalla de uno de los entornos creados para el desarrollo de la herramienta.

¹Enlace a la documentación oficial de Unreal sobre los árboles de comportamiento: <https://dev.epicgames.com/documentation/unreal-engine/behavior-trees-in-unreal-engine>

3.2.1. Tareas y condicionales

Junto con el agente mencionado anteriormente se han preparado una serie de tareas (los cuales servirán como nodos terminales de los árboles) y condicionales que sirven como base para poder crear ciertos comportamientos. Estas tareas y condicionales son introducidos en una clase implementada mediante el patrón Abstract Factory con el fin de simplificar la futura construcción de instancias de cada nodo. Para la construcción de los nodos mediante estas clases se precisa de una ID que identifique a la tarea o al condicional y una descripción de lo que hace o verifica, la cual será fundamental para el contexto del LLM. En esta descripción, para las clases que requieran una variable de la Blackboard, es recomendable introducir qué tipo de variable se requiere para que el modelo LLM sea más preciso.

Las tareas, junto con sus descripciones en español (en el trabajo realizado las descripciones han sido escritas en inglés para respetar el idioma del prompt) que se han creado para estas pruebas son las siguientes:

- **RotateToFaceBBEntry**: Rotar hasta encarar objetivo. El objetivo tiene que ser de tipo “Objeto”.
- **ChasePlayer**: Perseguir al jugador.
- **MoveTo**: Mover a la posición u objeto de destino. La variable objetivo tiene que ser de tipo “Objeto” o “Vector3”
- **FindRandomPatrol**: Elegir un punto aleatorio del mapa y guardarlo como objetivo.
- **Wait**: No hacer nada.
- **PickRedFlower**: Coger una flor roja.
- **PickBlueFlower**: Coger una flor azul.
- **PickYellowFlower**: Coger una flor amarilla.
- **PickBlackFlower**: Coger una flor negra.
- **ChooseRedFlower**: Elegir una flor roja como objetivo.
- **ChooseBlueFlower**: Elegir una flor azul como objetivo.
- **ChooseYellowFlower**: Elegir una flor amarilla como objetivo.
- **ChooseBlackFlower**: Elegir una flor negra como objetivo.
- **Attack**: Atacar si hay algún objetivo cerca.
- **SelectWaypoint**: Elegir un punto de ruta aleatorio y establecerlo como objetivo. El objetivo debe ser de tipo “Objeto”

Los condicionales implementados, junto con sus descripciones en español (al igual que las tareas, las descripciones en el trabajo están en inglés), son los siguientes:

- **HasLineOfSight?:** Cierto si el jugador está en línea de visión, falso si no. La variable que verifica tiene que ser de tipo “Bool”
- **IsItNear?:** Cierto si la distancia entre el ejecutor y el jugador es menor o igual que un umbral prefijado, falso en caso contrario. El objetivo tiene que ser de tipo “Objeto”.

Aunque el número de acciones y decoradores disponibles en esta implementación es reducido sirve como sistema de pruebas para la generación y la propuesta de evaluación.

3.2.2. Formato del árbol de comportamiento generado

Para la generación del árbol de comportamiento se ha decidido que el formato que debe devolver la herramienta generada sea un archivo JSON debido a su facilidad para leerlo, tanto por un humano como por un programa. Para la futura traducción del JSON a un árbol de Unreal ha sido necesaria la estandarización del formato del JSON. Para ello se ha hecho uso del JSON Schema Draft 2020-12², el cual es un lenguaje declarativo que sirve para definir una estructura de datos en formato JSON para que pueda ser considerada válida.

Este JSON Schema consta de dos partes: la Blackboard y el árbol de comportamiento. La Blackboard es una lista de objetos JSON que contienen un par clave-valor, donde la clave es el nombre de la variable y el valor es el tipo de la variable (Bool, Int, Float, Vector, Object o String). El árbol consta de un nodo (la raíz) que debe contener el tipo de nodo (Task, Sequence, Selector o Parallel), una lista de decoradores (en el caso de no tener se debe quedar vacía), la cual consiste en objetos JSON que debe contener como clave el nombre del decorador y como valor la variable de la Blackboard que va a usar, y una lista de nodos hijos. Estos nodos hijos siguen la misma estructura que la raíz. Como propiedades adicionales, puede contener una lista de nombres de variables de la Blackboard que vaya a necesitar, y, en el caso de ser una tarea, debe tener el ID de ésta. Este esquema se puede encontrar en el apéndice A

Para cuando el modelo haya generado el JSON con el árbol de comportamiento, el escenario entra en el modo de juego de forma automática. Es en este momento donde el NPC, desde su función “BeginPlay” (función de Unreal que es llamada cuando se inicia el juego), coge el nombre del archivo JSON generado para traducirlo a la lógica de Unreal. Esta traducción la hace la clase “BTConstructor”, la cual es una clase que sigue el patrón Singleton con el fin de que pueda ser accedida desde cualquier clase del juego. Esta clase crea un árbol de comportamiento de Unreal (clase “BehaviorTree”) y una Blackboard (clase “BlackboardData”) que serán rellenados con la información del JSON. Para ello, la clase rellena primero la Blackboard creando las variables disponibles en el JSON. Finalmente se crea el árbol mediante un método recursivo que recorre y genera el árbol en profundidad, creando primero todos los nodos hijos de una rama antes de pasar a sus hermanas.

²documentación del JSON Schema Draft 2020-12: <https://json-schema.org/draft/2020-12>

Una vez se ha especificado todo lo necesario para la generación de los árboles se ha procedido a la conexión con la herramienta que los genera. Como se ha mencionado en la sección 3.1, por comodidad del usuario es el propio entorno quien lanza el proceso de Python, haciendo de la generación un modelo de caja negra para el usuario. Para ello se lanza el proceso Python con el archivo principal de la herramienta y se le pasan como argumentos la ruta del script de la herramienta, las listas de tareas (la lista de tareas que requieren de una variable de la Blackboard y la lista de las que no por separado), la lista de decoradores, el comportamiento buscado y la lista de entradas de la Blackboard.

3.3. Generación de los árboles de comportamiento

3.3.1. Generación de forma directa

Se planteó la generación de los árboles de comportamiento de forma directa, en la que se le pasaba el prompt del usuario a un modelo local y éste generaba un BT en el formato JSON especificado en el apéndice A. Este planteamiento se realizaba mediante el módulo de Python “guidance”, el cual permite forzar un formato de salida a un modelo de Ollama³, el cual es un gestor y motor de ejecución de LLMs. Para este caso se probaron diferentes modelos, todos probados bajo el mismo escenario y con la temperatura en 0.0:

- **Prompt:** NPC that, constantly, select a random point from the map, goes to the point and waits a time.
- **Acciones disponibles:** ChooseRandomPoint, MoveTo, WaitTime, RotateToFaceBBEntry.
- **Variables de la Blackboard:** Point (Vector3)

Se buscó una tarea sencilla para establecer un baseline de los modelos. Los modelos probados para este planteamiento, con sus parámetros, su tiempo de ejecución y su resultado se pueden ver en la tabla 3.1:

Tabla 3.1: Modelos probados para la generación directa del JSON mediante guidance.

Modelo	Parámetros	Tiempo de ejecución
microsoft/Phi-4-mini-instruct	4B	10 segundos
qwen2.5:7b	7B	20 segundos
Qwen/Qwen2.5-Coder-7B-Instruct	7B	30 minutos
llama3.1:8b	8B	2 horas

Continúa en la siguiente página

³documentación oficial de Ollama: <https://ollama.com/>

Tabla 3.1: Modelos probados para la generación directa del JSON mediante guidance. (Continuado)

Modelo	Parámetros	Tiempo de ejecución
deepsek-r1:8b	8B	>2 horas

Los modelos de menor tamaño no generaban buenos árboles para una tarea tan sencilla (como tareas que faltaban o variables de la Blackboard mal asignadas). Sin embargo, los modelos grandes si conseguían generar árboles que cumplían con la función que se les pedía, aunque añadiendo más tareas de las necesarias. Pero, como se puede observar en la tabla, para conseguirlos la ejecución podía tardar horas, por lo que también debían ser descartados. Tanto la malfuncionalidad de los modelos pequeños como la tardanza de los modelos grandes se puede explicar por el funcionamiento de guidance y la complejidad recursiva del JSON Schema empleado, por lo que esta primera generación debería ser descartada.

3.3.2. Generación mediante fragmentación por subtareas

En base a otros trabajos relacionados se replanteó la generación para dividirlo en subtareas con el fin de simplificar la tarea al LLM ([Hoffmeister et al. \(2026\)](#) y [Choi et al. \(2026\)](#)). Para ello se encargaban cuatro agentes LLM, cada uno con un rol distinto. El primero se encargaba de dividir la acción mandada por el usuario mediante el prompt en subtareas. En el ejemplo de un NPC que tiene que buscar al jugador para perseguirlo y atacarlo cuando esté cerca, este primer agente debería dividir la tarea principal en:

1. Buscar al jugador
2. Perseguir al jugador
3. Comprobar la distancia del NPC al jugador
4. Atacar al jugador

Una vez están separadas las tareas en subtareas, un segundo agente selecciona para cada subtask el subconjunto de las acciones necesarias para realizar esta subtask. Estas acciones que selecciona se guardan en una lista directamente en el formato JSON especificado por el JSON Schema. Cuando ya se ha seleccionado el subconjunto un tercer agente decide qué tipo de nodo padre van a tener los subconjuntos (Selector, Sequence o Parallel). Finalmente un último agente revisa los decoradores y las tareas que requieran de una variable de la Blackboard para adjudicársela. Se puede ver un diagrama de flujo de esta generación en la figura 3.3.

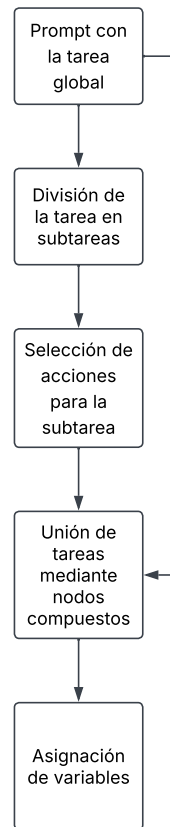


Figura 3.3: Diagrama de flujo de la generación de BTs mediante división en subtareas

Finalmente se descartó esta opción debido a que los árboles resultantes quedaban muy planos (con menos niveles de profundidad de lo que se cabía esperar), haciéndolos difícilmente legibles y modificables. Además los agentes tendían a equivocarse (mala división en subtareas, confusión entre el tipo que debería ser un nodo padre) o a alucinar (inventarse tareas o decoradores que no existen), no consiguiendo de este modo un árbol de comportamiento válido con ninguno de los modelos probados.

3.3.3. Generación mediante pseudocódigo

Algunas investigaciones de los LLMs muestran que, dado un conjunto de acciones posibles a realizar, los LLMs son capaces de seleccionar de éstas un orden para realizar una tarea de alto nivel ([Ahn et al. \(2022\)](#) y [Song et al. \(2023\)](#)). Es por esto que se decidió que el LLM generase esta secuencia en un formato de pseudocódigo que fuese fácilmente convertible al formato JSON deseado. Este pseudocódigo se especifica de la siguiente manera:

- Las palabras reservadas son: SEQUENCE, SELECTOR, PARALLEL, IF, THEN, ELSE, DO, END.
- Los bloques de nodos compuestos (como SELECTOR) deben acabar con la palabra END.

- Las condiciones deberán ser escritas como “IF (nombre del condicional de la lista de decoradores) THEN”.
- Los condicionales serán usados por el nodo siguiente.
- Las acciones deberán ser escritas como “DO (nombre de la acción de la lista de acciones)”.

Se puede ver un ejemplo de este pseudocódigo en la figura 3.4.

```
SELECTOR
  IF has_key THEN
    SEQUENCE
      DO move_to_door
      DO open_door
    END
  ELSE
    SEQUENCE
      DO find_key
      DO move_to_door
      DO open_door
    END
  END
END
```

Figura 3.4: Ejemplo del pseudocódigo que el LLM debe generar.

Como se ha podido ver en trabajos relacionados, la generación de árboles de comportamiento mediante LLMs mejora considerablemente en modelos a los que se le ha aplicado fine-tuning o se les proporciona ejemplos (Lykov y Tsetserukou (2023)). Debido a la falta de capacidad computacional no se ha podido hacer un Fine-tuning de un LLM, por lo que se ha optado a recopilar una pila de ejemplos de BTs para poder hacer un sistema RAG. Este es una técnica de inteligencia artificial que consiste en conectar un LLM con una base de datos externa a sus conocimientos disponibles que puede consultar para ampliar su contexto mediante búsquedas de conceptos próximos en el espacio vectorial (generalmente utilizando la distancia coseno de dos vectores como métrica) al prompt dado, posibilitando así la reducción de alucinaciones.

Para ello se ha escogido el dataset publicado por Lykov y Tsetserukou (2023)⁴, el cual contiene 8366 ejemplos de árboles de comportamiento generados de forma sintética. Debido a que estos árboles están en formato XML no se pueden utilizar de forma predeterminada para el sistema RAG, por lo que se ha procedido a hacer un preprocesamiento de los datos.

Este preprocesado carga el dataset desde Hugging Face (plataforma de código abierto donde los usuarios pueden subir modelos o datasets, similar a otras como Kaggle), los limpia y los traduce al formato necesario. La limpieza consiste en el uso

⁴Enlace a la página de Hugging Face: https://huggingface.co/datasets/ArtemLykov/LLM_BRAIn_dataset

de expresiones regulares para poder subsanar los fallos de formato más comunes que tenían estos datos, como diferencias entre nombres por mayúsculas y minúsculas, etiquetas sin la apertura o la clausura, faltas de comillas, etc. Si hay algún ejemplo que, aun con la limpieza, no se ha conseguido parsear por otro fallo, se descarta. Finalmente, de los 8366 ejemplos se quedaron 8290 árboles válidos.

Una vez los datos están limpios, se procede a la traducción del formato XML gracias al módulo “xml.etree.ElementTree”, el cual es un módulo de Python que sirve para trabajar con datos que estén en formato XML. Cuando el árbol ha sido construido se guarda en formato JSON siguiendo el JSON Schema especificado para este trabajo, y finalmente se traduce al pseudocódigo especificado previamente gracias a otra función recursiva, quedando así 8366 archivos JSON con el prompt de la acción de alto nivel a realizar, el árbol de comportamiento resultante en formato JSON y el pseudocódigo.

Con los ejemplos de los árboles de comportamiento preparados se ha procedido a la aplicación del sistema RAG. Al iniciar la herramienta de generación se crea el almacén vectorial cargando enteros los documentos mencionados anteriormente en estructuras llamadas “Document”, provenientes del módulo “langchain_core.documents”. Una vez están todos los documentos cargados se procede a hacer el embedding de éstos de manera local gracias al modelo de Hugging Face “all-MiniLM-L6-v2” mediante el módulo “langchain_huggingface”. Estos embeddings se crean y se almacenan en una estructura de tipo FAISS gracias al módulo “langchain_community.vectorstores”. Una vez creado el almacén vectorial se rescatan las 5 búsquedas más similares al comportamiento pedido por el usuario y se incluyen estos ejemplos (prompt con la descripción del comportamiento buscado y pseudocódigo del árbol) en el prompt del sistema del LLM.

Para la generación de pseudocódigo a partir del comportamiento buscado se ha requerido un modelo de mayor tamaño que los previamente usados debido a la complejidad del problema, por lo que se ha usado el LLM “Mistral Large 3”, el cual cuenta con un total de 675B de parámetros (41B de ellos son parámetros activos, los que actúan por cada token), mediante el módulo “langchain_mistralai”, el cual realiza llamadas de API al modelo. Para el prompt de sistema del modelo se han realizado varias técnicas de *prompt engineering* con el fin de reducir las alucinaciones en la generación del pseudocódigo:

- **Role prompting:** Se le especifica al LLM como debe actuar: “You are an expert in behavior trees and algorithm design.”.
- **Especificación de la tarea:** Se le especifica para qué tarea debe responder: “Your task is to convert a high-level task description into a structured pseudocode algorithm that can later be transformed into a Behavior Tree.”.
- **Few-shot prompting:** Se le especifican varios ejemplos, sacados del sistema RAG mencionado anteriormente.
- **Chain-of-Thought:** Obliga al modelo a, internamente, seguir la línea de pensamiento representada en la figura 3.5

- **Self-Critique Prompting:** Le pide al modelo que evalúe su respuesta buscando si se ha inventado acciones o condicionales. Esto se hace en una segunda llamada con el siguiente prompt: “Verify if your pseudocode is made only with the actions and conditions available. Responde me with ONLY the final pseudocode and no explanations or plain text”

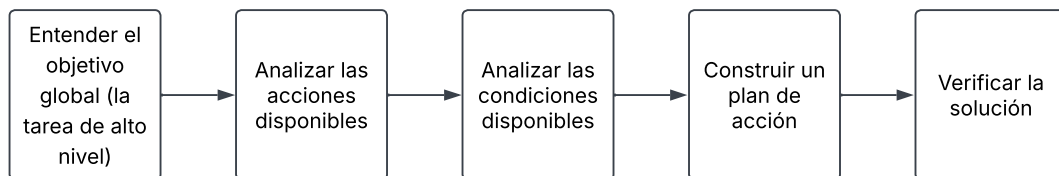


Figura 3.5: Diagrama de la cadena de pensamiento que sigue el LLM generador de pseudocódigo.

El prompt de sistema completo se puede ver en el apéndice [B](#).

Una vez se ha generado el pseudocódigo se procede a la traducción de éste al formato JSON. Esto se hace mediante la clase “BehaviorTreeParser” y mediante dos clases auxiliares: Composite y Action, las cuales ambas heredan de la clase Node. La clase Composite contiene el tipo (un string), una lista de nodos hijos (lista de tipo “Node”), un decorador (opcional de tipo string), las variables de la Blackboard del decorador (una lista de strings) y un identificador (un entero). La clase Action tiene el nombre de la tarea (un string), las variables de la Blackboard que puede usar la tarea (una lista de strings), un decorador (opcional de tipo string), las variables de la Blackboard del decorador (una lista de strings) y un identificador (un entero). Para la traducción del pseudocódigo primero se preprocesa guardando cada línea en una lista de strings. Seguido de esto se procesa cada línea dividiendo cada palabra de ésta y actuando dependiendo de cuál es su primera palabra:

- **SEQUENCE, SELECTOR o PARALLEL:** Se crea un nodo de tipo “Composite”, se establece su tipo como la palabra usada y se verifica si hay una condición pendiente. Si es el caso, se establece esta condición. Además, se añade este nodo a su padre (en el caso de haberlo es el último nodo añadido a una pila auxiliar) y se añade a la pila auxiliar.
- **IF:** Se verifica si tiene un condicional o si el condicional que tiene existe. En caso contrario de cualquiera de los dos se lanza un “ParserError” indicando la línea y el fallo. En el caso de que no haya error se establece la condición como la condición pendiente.
- **DO:** Se verifica si tiene una acción o si esa acción existe. Al igual que en el caso anterior, de no ser así se lanza un “ParserError”. En caso de haberlo se crea un nodo de tipo “Action” con el nombre de la acción que hay a continuación. Igualmente que con los nodos compuestos, se verifica si hay alguna condición pendiente y se añade el nodo al padre.

- **END:** Se verifica que la pila no esté vacía. En caso contrario significa que hay un “END” sin bloque que cierre, por lo que se lanza un “ParserError”. En caso de que sí esté vacía se saca un nodo de la pila. A continuación se verifica si la pila se ha quedado vacía, lo que significaría que ese nodo era la raíz del árbol. En caso de ya haber raíz se lanza un “ParserError” indicando que hay múltiples raíces.

Una vez se acaba este procesamiento de cada línea se verifica si sigue habiendo pila (lo que indicaría que hay bloques sin cerrar) o si no se ha generado una raíz. Los posibles “ParserError” lanzados durante este proceso se recogen fuera de esta función para poder volver llamar al LLM indicando el error de la excepción con el fin de que devuelva un pseudocódigo arreglado.

Cuando ya ha sido procesado todo el pseudocódigo se crea una clase auxiliar llamada “BlackboardAssignmentAgent” que se encarga de asignar las variables de la Blackboard a las tareas o los decoradores. Para ello reconstruye el pseudocódigo asignándole el identificador a cada nodo (este identificador es el orden de aparición de cada acción o decorador, ya que en los árboles de comportamiento es normal que se repitan) y se le pide a un segundo LLM que a cada decorador y acción que necesite una variable de la Blackboard se le asigne. Por problemas con sobrecarga de peticiones de la API, para este caso se usa un modelo local más pequeño, el modelo “llama3:8b”. Para el prompt de sistema de este LLM se le pasa el árbol en el formato del pseudocódigo reconstruido mencionado anteriormente, la lista de las acciones que necesitan una variable de la Blackboard con sus descripciones y la lista de variables de ésta. Para disminuir las alucinaciones de este LLM se han usado las siguientes técnicas de *prompt engineering*:

- **Role prompting:** “You are an expert in Unreal Engine Behavior Trees.”
- **Especificación de la tarea:** “Your task is NOT to understand the whole game. Your ONLY task is assigning Blackboard variables to Behavior Tree nodes.”
- **One-shot prompting:** Se le especifica un ejemplo inventado mostrado en la figura 3.6

Una vez genera la respuesta se valida el resultado comprobando si todos los nombres (variables y acciones) son correctos. Si hay algún nombre que no existe porque el modelo ha alucinado se le pide al modelo que lo reintente. Cuando el resultado es válido se guardan las variables asignadas en los nodos correspondientes y, finalmente, se construye el JSON mediante una función recursiva desde la raíz del árbol y se guarda en un archivo con el nombre especificado por el usuario.

Algunos trabajos han demostrado que la planificación generada por LLMs como se ha hecho en este trabajo no resultan ser fiables de forma exacta, por lo que se ha diseñado un sistema de validación y corrección de errores automático (Valmeekam et al. (2023)).

Example Input:

Actions : grab_key : Stores the key in the inventory,

move_to : Moves to a position,

open_door: Open a door

Variables: ["Key" : "Object", "Door" : "Object", "Enemy_position" : "Vector3"]

Tree:

SEQUENCE

DO move_to#1

DO grab_key#1

DO move_to#2

DO open_door#1 [Decorator=IsNear?#1]

DO wave#1

Output format:

```
{
  "Actions": {
    "move_to#1": ["Key"],
    "grab_key#1": ["Key"],
    "move_to#2": ["Door"],
    "open_door#1": ["Door"]
  },
  "Decorators": {
    "IsNear?#1": ["Door"]
  }
}
```

Figura 3.6: Ejemplo del input y el output del LLM que establece las variables de la Blackboard.

3.4. Validación automática de los árboles de comportamiento

Para que los árboles de comportamiento generados se considerasen válidos para las pruebas se han tenido en cuenta dos factores: que los árboles generados respeten el orden de los nodos (si hay acciones dependientes unas con otras no deben ejecutarse los hijos antes que los padres) y que deben cumplir un objetivo. Estas comprobaciones no son obligatorias para el funcionamiento de la herramienta, sino que es un apoyo para automatizar este proceso.

A continuación se describen los métodos de las validaciones previamente mencionadas.

3.4.1. Validación de dependencias

Para comprobar el orden correcto de los nodos se ha recurrido al uso de un autómatas finito no determinista, NFA por las siglas en inglés de Non-Deterministic Finite Automata ([Park et al. \(2004\)](#), [Camacho et al. \(2017\)](#)). Este es un modelo matemático que procesa una secuencia de símbolos que, en cada paso, puede estar simultáneamente en varios estados (por eso no es determinista) ya que la transición desde un estado a otro con un símbolo dado puede no ser única. Este modelo acepta una secuencia si existe al menos un camino de transiciones válidas que consume toda la entrada sin fallar. En el caso del trabajo está definido de la siguiente manera:

- Los estados son los nodos del árbol de comportamiento.
- Las transiciones son las relaciones padre-hijo.
- El alfabeto son las IDs de las tareas.
- Hay un conjunto de estados abiertos.
- Hay una serie de estados iniciales.
- La función de transición es no determinista porque un mismo símbolo puede activar múltiples nodos de forma simultánea, ya que diferentes tareas pueden estar en diferentes puntos del árbol.

Este NFA está implementado en la clase “BehaviorList”, la cual está implementada mediante el patrón Singleton para que todas las tareas puedan acceder a ella. Para esta verificación se necesita un archivo JSON en el que se especifican las restricciones. Se puede ver un ejemplo de un JSON con las restricciones en el apéndice [C](#).

Una vez se construyan las dependencias mediante el NFA empieza la ejecución del árbol. Para que funcione esta comprobación a la hora de ejecutarse una tarea o un decorador, éste debe llamar a la función “addBehavior” de esta clase con el nombre de la tarea. Cuando llega una tarea nueva se realiza la comprobación de las dependencias de la siguiente forma:

1. Se recuperan los nodos candidatos gracias a su ID.

2. Se crea un nuevo conjunto de nuevos estados.
3. Si un candidato no tiene padre (no depende de ninguna otra acción) se añade este nodo a los nuevos posibles estados.
4. Se recorren todos los hijos de los estados activos y se comprueba si coinciden con el ID de la tarea o el decorador actual. En caso afirmativo se añade este hijo al conjunto de nuevos estados.

Al final de la función se comprueba si el conjunto de nuevos estados está vacío. En el caso de que el conjunto de nuevos estados no está vacío implica que sí se han respetado las dependencias, por lo que éste pasa a ser el conjunto de estados abiertos. Por el otro lado, si el conjunto de nuevos estados está vacío es que no existe ningún posible estado en el que el orden de la ejecución de los nodos sea el actual, lo que implica que no se han respetado las dependencias y la validación falla.

Para los casos en los que la validación puede fallar, se ha creado una clase interfaz “IValidatorCallback” que se encarga de gestionar las terminaciones de la validación por fallos o por terminación de la ejecución. Ya que es la clase “GeneratorComponent” quien se encarga de llamar a la herramienta de generación, es esta quien hereda de la interfaz y quien es llamada por la clase “BehaviorList” cuando falla la validación, pasándole una lista de todas las acciones ejecutadas hasta el momento y la acción en la que ha fallado. Es entonces cuando la clase “GeneratorComponent” interrumpe el modo de juego y vuelve a llamar a la herramienta con el pseudocódigo fallido, el contexto dado por “BehaviorList” y el prompt con la acción original y vuelve a intentar la ejecución cuando la herramienta arregla el pseudocódigo.

3.4.2. Validación de objetivos

Para validar los objetivos específicos de los comportamientos se ha creado una clase interfaz “ObjectiveValidatorBase” que contiene un método para comprobar si el objetivo se ha cumplido y un mensaje de error indicando que el objetivo ha fallado. De esta interfaz deben heredar componentes de Actor para que los objetos “BTGeneratorManager” los puedan tener y puedan hacer la comprobación de los objetivos una vez se acabe la ejecución. Si la comprobación del objetivo es fallida se le pasa a la herramienta el mensaje de error y la lista de tareas ejecutadas, así como el pseudocódigo fallido y el prompt original, y se vuelve a intentar con un pseudocódigo arreglado.

3.4.3. Entornos creados para la validación

Una vez se han tenido los métodos de validación de los árboles de comportamiento se han creado tres entornos con diferentes pruebas.

El primero de ellos consta de tres flores de distintos colores: rojo, azul y amarillo. En este entorno se le pide a la herramienta que genere un árbol donde recoja una única flor de cada tipo, independientemente del orden, con el siguiente prompt: “NPC that has to choose and pick one flower red, one flower yellow, and one flower blue. In order to pick a flower it has to be first in the position of the flower.”. Al ser una

tarea sencilla la herramienta solo ha necesitado un intento y un tiempo de ejecución de 84.84 segundos para crear un árbol válido y que cumpla con el objetivo.

El segundo de los entornos consta de tres flores azules, una flor roja y una flor amarilla, y aquí se le pide a la herramienta que genere un árbol que recoja tres flores azules y después una roja mediante el siguiente prompt: “NPC that has to choose and then pick first three blue flowers, and, after that, has to choose and pick one red flower. In order to pick a flower it has to be first in the position of the flower”. Al igual que en el caso anterior, solo ha necesitado de un intento y un tiempo de 45.46 segundos para realizar un árbol válido. En la figura 3.7 se puede ver una captura de pantalla de este entorno.

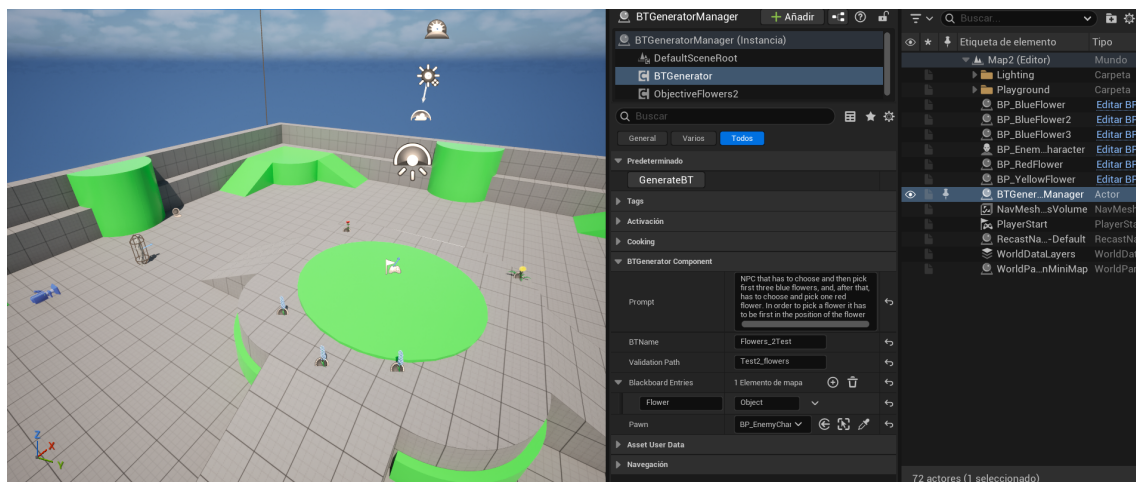


Figura 3.7: Captura de pantalla del segundo entorno de pruebas de validación.

El tercer entorno consta de un enemigo y un punto de patrulla. Aquí se le pide a la herramienta que genere un comportamiento clásico en videojuegos: un NPC que patrulle el escenario por distintos puntos y, que si ve al jugador, le persiga para atacarle. Para ello se ha usado el siguiente prompt: “NPC that has to do one of this two options: - If the objective is at his line of sight it has to: 1. chase him 2. move to his position 3. if is near the objective, it has to attack him. - Otherwise, it has to: 1. select a waypoint 2. move to the waypoint”.

A pesar de ser una petición más compleja debido a la naturaleza del comportamiento y al aumento de número de variables de la Blackboard (cuatro variables, dos condicionales para ver si el objetivo está en el punto de mira y si el objetivo está cerca, el punto de patrulla a elegir y el objetivo a perseguir), la herramienta ha sido capaz de generar en 153.1 segundos un árbol de comportamiento válido a la primera en cuanto a cumplimiento de objetivos y respetando las dependencias, aunque sí se ha tenido que modificar alguna tarea que requería de una variable de la Blackboard que el segundo LLM no le ha adjudicado ninguna variable. En la figura 3.8 se puede ver una captura de pantalla de este tercer entorno.

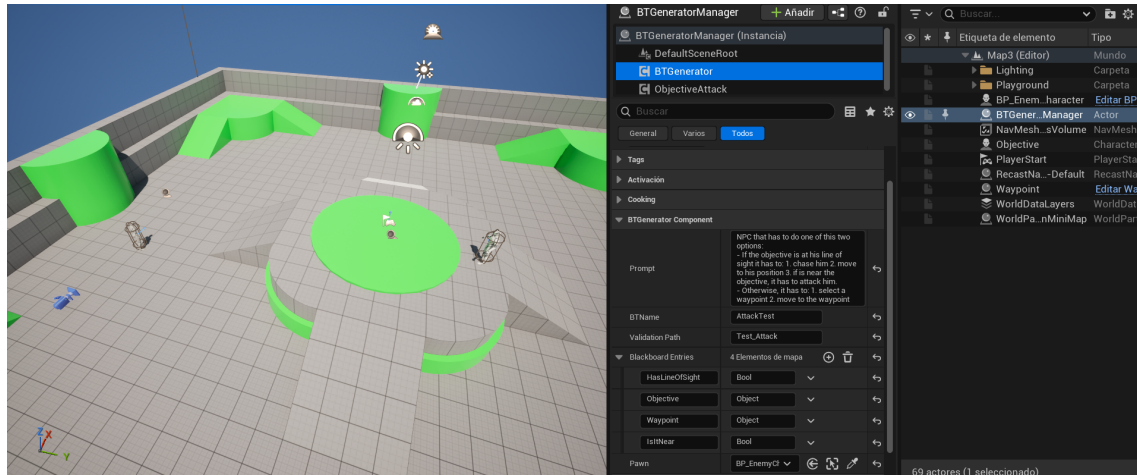


Figura 3.8: Captura de pantalla del tercer entorno de pruebas de validación.

Con los resultados de esta validación, resumidos en la tabla 3.2, se ha podido establecer un baseline del funcionamiento de la herramienta para, finalmente, realizar pruebas con usuarios.

Tabla 3.2: Resultados de las pruebas de validación

Entorno	Intentos	Tiempo de ejecución
1	1	84.84 segundos
2	1	45.46 segundos
3	1	153.1 segundos

Evaluación y resultados

Para validar que la herramienta cumple con los objetivos planteados se ha llevado a cabo el plan de usuarios descrito a continuación.

4.1. Pruebas con usuarios

Como se ha mencionado en la sección 1.2, lo que se busca en este trabajo es una herramienta capaz de generar de manera similar a los humanos un árbol de comportamiento funcional o, al menos, una aproximación que sea capaz de disminuir el trabajo a los diseñadores de videojuegos. Con este objetivo en mente se ha diseñado unas pruebas con usuarios para comprobar si se cumplen los objetivos planteados.

Para ello se ha realizado un plan de pruebas donde se evalúan los tiempos de ejecución de generación y ejecución de los árboles de comportamiento, la comparación entre los tiempos de un árbol generado por un humano y uno generado por la herramienta, los intentos de generación de un árbol y la calidad de la herramienta en cuanto a usabilidad y funcionalidad. Para estas métricas las hipótesis iniciales eran las siguientes:

- H1: La herramienta es capaz de generar árboles más rápido que un diseñador que los hace a mano
- H2: Los árboles generados realizan una ejecución similar a los realizados por un humano
- H3: La calidad de los árboles generados por la herramienta es un poco menor que los generados por un humano, pero les permite a los diseñadores tener un punto de partida más rápido que puede ser fácilmente modificable, disminuyendo así el tiempo de trabajo
- H4: La herramienta de generación de árboles es intuitiva y fácilmente de utilizar.

Para poder comprobar estas hipótesis se han buscado usuarios que cumplan con el perfil de un diseñador de videojuegos que haya trabajado con Unreal Engine y sus árboles de comportamiento. El procedimiento con los usuarios ha sido el siguiente:

1. Con el fin de mantener el anonimato, se le asigna un identificador único a cada usuario.
2. Se le pide que haga un árbol de un NPC que patrulla hasta que ve a un objetivo, que debe perseguirlo hasta estar lo suficientemente cerca como para atacarlo.
3. Se le pide que haga un árbol de un NPC que va recogiendo flores hasta que ve a un objetivo, que en ese caso debe huir a un punto del mapa.
4. Se les cronometra desde el momento en el que se les indique que pueden empezar hasta que consigan una ejecución que satisfaga a los objetivos propuestos.
5. Se les pide que realicen los mismos árboles con la herramienta.
6. Se les pide que rellenen un formulario sobre la usabilidad y calidad de la herramienta.

Para cada árbol que debería ser creado se hicieron dos escenarios distintos, uno en el que se deba usar la herramienta y otro en el que el NPC ejecuta un árbol creado a mano a través del sistema de Blueprints (sistema de programación visual de Unreal). Para el primer árbol se ha hecho una copia del tercer escenario mencionado en la sección 3.4, con un objetivo y un punto de patrulla cerca de él en el mapa. El segundo escenario consta de un objetivo, dos flores rojas (una de ellas en frente del objetivo para que el NPC tenga que pasar por delante de él) y un punto de patrulla alejado del objetivo.

El formulario, que se puede encontrar en el apéndice D, mide la usabilidad y la calidad de la herramienta frente a BTs creados por un humano. Para la usabilidad se ha decidido evaluar mediante el System Usability Scale (SUS), que consiste en un cuestionario estandarizado de 10 items que siguen la escala de Likert de 5 puntos, que van desde “Totalmente en desacuerdo” a “Totalmente de acuerdo” (Brooke (1996)). Este método de evaluación se ha decidido seguir por evidencias de su robustez (Bangor et al. (2008) y Peres et al. (2013)).

Para la calidad de la herramienta se les han realizado preguntas sobre los resultados esperados de tiempos de ejecución, diferencias entre la calidad y similitud de sus árboles de comportamiento y los generados por la herramienta, y su funcionalidad como punto de partida. Finalmente se ha habilitado un campo de texto no obligatorio de respuestas libres donde los usuarios podrían expresar su opinión sobre los árboles generados y la herramienta.

4.2. Resultados de las pruebas de usuario

Debido al perfil específico que se necesitaban para las pruebas solo se han podido realizar pruebas con 3 usuarios, por lo que las conclusiones sirven como una aproximación pero no se deben tomar como concluyentes.

4.2.1. Resultados de la usabilidad de la herramienta

En cuanto a la usabilidad podemos ver los resultados individuales en la tabla 4.1, donde se ha obtenido una media de 90.83 sobre 100 con una desviación estándar de 15.88, lo cual supera con creces la media de 68 establecida en la literatura, por lo que dadas estas pruebas se puede decir que la herramienta cumple con el objetivo de accesibilidad, haciéndola así fácil de utilizar y confirmando con los datos dados H4 (Hyzy et al. (2022)).

Tabla 4.1: Resultados de la encuesta SUS

	Puntuación
U1	72.5
U2	100
U3	100
Media	90.83
Desviación estándar	15.88

4.2.2. Resultado de la comparativa de tiempos

La comparativa de los tiempos podemos verla en la tabla 4.2 y en la figura 4.1, en donde a excepción del primer árbol de comportamiento que ha tardado más en generarlo la herramienta que el segundo usuario, hay una diferencia significativa en cuanto a la tardanza, beneficiando a los generados por la herramienta. Además, como podemos ver en la tabla 4.3, los usuarios tienen una opinión favorable sobre los tiempos de ejecución de la herramienta con una media del 86.6 % y una desviación típica de 0.58, afirmando así la hipótesis inicial de generar de forma más rápida los árboles de comportamiento con unos tiempos de generación adecuados (H1).

Tabla 4.2: Tiempo (en segundos) de generación de cada árbol de comportamiento por cada usuario

Usuario	Árbol 1 manual	Árbol 1 herramienta	Árbol 2 manual	Árbol 2 herramienta
U1	578	238	332	142
U2	315	486	159	35
U3	271	35	174	37

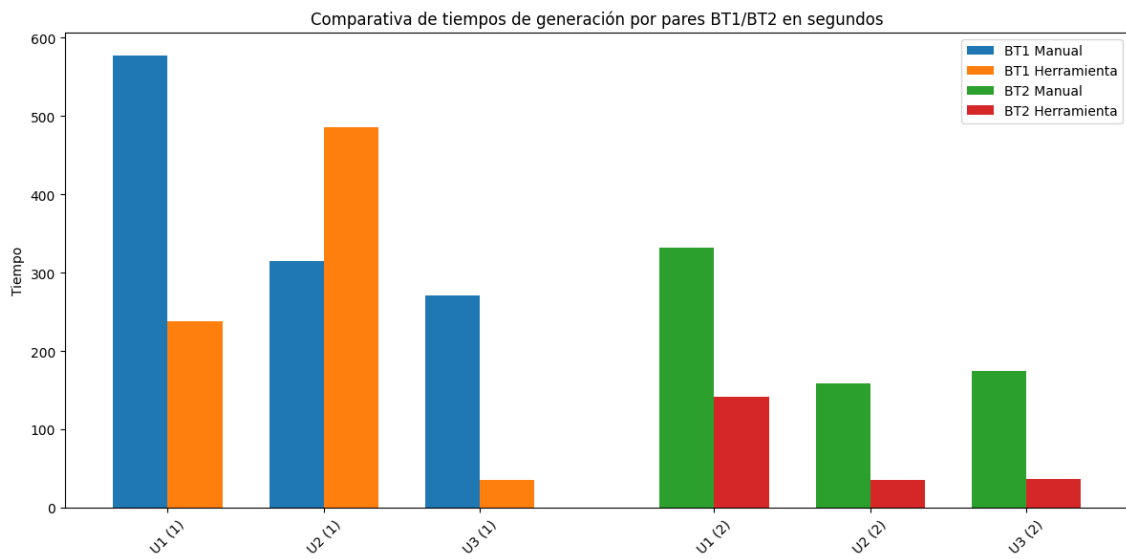


Figura 4.1: Comparación de cuánto han tardado en crear los usuarios los árboles de comportamiento junto a cuánto ha tardado la herramienta.

Tabla 4.3: Opinión de los usuarios sobre los tiempos de ejecución de la herramienta generadora de árboles de comportamiento del 1 al 5, siendo 1 una opinión desfavorable y 5 la más favorable.

	Puntuación
U1	5
U2	4
U3	4
Media	4.33
Desviación estándar	0.58

4.2.3. Resultados de la calidad y disminución de carga de trabajo

Siguiendo con la calidad de los árboles de comportamiento, podemos observar en las tablas 4.4 y 4.5 que la calidad de los árboles es inferior a los generados por un humano (con una media en las respuestas del 80 % y una desviación típica de 1), pero que cumplen con la ejecución esperada (con una media de las respuestas del 80 % y una desviación típica de 0). Esto se puede explicar gracias a las respuestas

de las preguntas sobre si los árboles generados tenían más nodos de los necesarios y si eran distintos a los generados por los humanos, en los que todos los usuarios respondieron que la herramienta añadía algunos nodos extra (tanto compuestos como tareas innecesarias) y que esto hacía que los árboles generados fuesen distintos a los creados manualmente, dando una calidad inferior pero con una ejecución similar, confirmando así la hipótesis H2.

Tabla 4.4: Respuestas de los usuarios sobre la calidad de los árboles de comportamiento generados por la herramienta, donde 1 es que no están de acuerdo con la afirmación de que la calidad de los árboles generados por la herramienta es inferior y 5 es que están completamente de acuerdo.

	Puntuación
U1	5
U2	3
U3	4
Media	4.00
Desviación estándar	1.00

Tabla 4.5: Respuestas de los usuarios sobre la ejecución esperada de los árboles de comportamiento, donde 1 es que consideran una mala ejecución y 5 significa una ejecución correcta.

	Puntuación
U1	4

Continúa en la siguiente página

Tabla 4.5: Respuestas de los usuarios sobre la ejecución esperada de los árboles de comportamiento, donde 1 es que consideran una mala ejecución y 5 significa una ejecución correcta. (Continuado)

	Puntuación
U2	4
U3	4
Media	4.00
Desviación estándar	0.00

Finalmente y junto a los resultados del apartado de calidad, la comprobación de H3 se puede apoyar en los resultados de las tablas 4.6, 4.7 y 4.8, donde podemos ver que los usuarios han tenido una buena opinión sobre la idea de la utilidad de la herramienta como apoyo para los diseñadores, respondiendo con un 93.4 % de media y una desviación típica de 0.58 a la opinión de que sirve como buen punto de partida y que, gracias a la facilidad de modificación de los JSONs generados (con una media de respuestas del 86.6 % y una desviación típica de 1.15), la herramienta puede servir para disminuir la carga de trabajo de los diseñadores (con una opinión de media del 93.4 % y una desviación típica de 0.58).

Tabla 4.6: Respuestas de los usuarios donde indican si los árboles de comportamiento generados sirven como punto de partida (5) o no (1)

	Puntuación
U1	5
U2	4
U3	5
Media	4.67
Desviación estándar	0.58

Tabla 4.7: Respuestas de los usuarios sobre la facilidad para modificar los árboles de comportamiento generados, donde 1 significa que es difícil y 5 significa que es fácil.

	Puntuación
U1	3
U2	5
U3	5
Media	4.33
Desviación estándar	1.15

Tabla 4.8: Respuestas de los usuarios sobre su opinión de si la herramienta puede disminuir (5) o no (1) la carga de trabajo de los diseñadores.

	Puntuación
U1	4
U2	5
U3	5
Media	4.67
Desviación estándar	0.58

Discusión

Este trabajo ha consistido en la creación de una herramienta para la generación automática de Behavior Trees (BTs) para personajes no jugador (NPCs) en videojuegos con el fin de disminuir la carga de trabajo de los diseñadores de videojuegos. Para ello se escogió el motor de videojuegos Unreal Engine 5.7 como base donde poder implementar nodos necesarios para la generación de comportamientos (tanto como tareas como condicionales) y donde desarrollar una serie de escenarios para probar los árboles de comportamiento generados. Una vez se tenía todo lo necesario para poder crear y ejecutar árboles de comportamiento se ha procedido con el diseño de la herramienta que los genera. Esta herramienta consta de dos LLMs: el primero, de un tamaño elevado (Mistral Large 3, con 675B de parámetros totales y 41B de parámetros activos), traduce la petición en lenguaje natural a pseudocódigo, mientras que un segundo LLM de menor tamaño (llama3:8B, con 8B de parámetros) se encarga de asignar las variables de la Blackboard a los nodos necesarios. Para evaluar los árboles se han diseñado dos sistemas de validación: un validador de dependencias entre nodos mediante un autómata finito no determinista y un validador de objetivos cumplidos. Finalmente se han realizado pruebas de usuario para verificar si se cumplen los objetivos planteados.

A continuación se analizan e interpretan los datos expuestos en la sección 4.2, evaluando el desempeño de la tecnología empleada, la efectividad del RAG y las implicaciones de las pruebas con usuarios con respecto a los objetivos originales del proyecto, así como las limitaciones que se han presentado durante el desarrollo del proyecto.

5.1. Discusión sobre la herramienta generadora de árboles de comportamiento

Se ha probado con distintos métodos de generación de los árboles de comportamiento.

Se empezó probando con un sistema que generase los árboles de forma directa en los que se probaron diferentes modelos locales mediante guidance, donde los más pequeños (4B y 7B de parámetros) generaban árboles incorrectos y los más grandes

(8B de parámetros) tardaban minutos o incluso horas en generarlos. Esto se puede explicar debido a la complejidad recursiva del JSON Schema, teniendo que descartar este método.

El segundo intento consistió en fragmentar el comportamiento en subtareas, pero esto ocasionó unos árboles demasiado planos, convirtiéndolos en ilegibles y difícilmente modificables y escalables. Además, la selección de subtareas resultó ser imprecisa.

Finalmente la decisión de emplear un pipeline híbrido con Mistral Large 3 (675B de parámetros totales, 41B de parámetros activos) en la nube y llama3:8B localmente demuestra un balance aceptable entre coste, latencia y precisión. El modelo en la nube proporcionó tiempos de procesamiento de pseudocódigo altamente eficientes con una baja latencia en las peticiones. No obstante, la delegación de la asignación de variables de la Blackboard al modelo local de menor tamaño (8B) causó errores esporádicos en los que ciertas tareas quedaban sin variables asignadas (como se evidenció de forma controlada en el entorno 3 de validación). Este comportamiento indica que, si bien se evita la sobrecarga y saturación de la API de terceros, los modelos locales de 8B aún carecen de la robustez necesaria para manejar de manera completamente precisa la lógica recursiva de las variables del árbol de comportamiento.

En cuanto al sistema RAG creado, a pesar de utilizar un dataset enfocado en robótica debido a la ausencia de datasets de videojuegos disponibles, el sistema cumplió con creces el objetivo de mitigar las alucinaciones del modelo. Al proveer al modelo de un contexto estructurado en el formato de pseudocódigo adecuado, este generó árboles con una cantidad de nodos extra reducida y mayor acierto en el uso de los tipos de nodos de composición. Su principal inconveniente fue de carácter temporal: en instancias donde el almacén vectorial FAISS no se encontraba precargado en memoria, el tiempo de inicialización de la herramienta se prolongó algunos segundos, afectando negativamente la experiencia inmediata del usuario.

Por otro lado, la sinergia de las técnicas de prompt engineering demostró ser esencial para la viabilidad de la generación de pseudocódigo. En particular, el Self-Critique Prompting actuó como un filtro crítico que previno la invención de identificadores de tareas y corrigió de forma autónoma defectos en la sintaxis antes de pasar el texto al traductor a JSON. Esto redujo de forma significativa los errores de formato.

5.2. Discusión sobre la validación

La inclusión del NFA para validar el orden secuencial de dependencias y de los validadores de objetivos representó una estrategia exitosa de control de calidad autónomo. Aunque su uso no es obligatorio para el funcionamiento básico de la herramienta, permite cerrar el ciclo de retroalimentación de la IA. Al ocurrir un fallo, la captura del contexto preciso (el pseudocódigo problemático y el historial de acciones ejecutadas) posibilita que el LLM reestructure de manera reactiva la lógica y logre corregir el comportamiento incorrecto sin necesidad de que el diseñador intervenga. Esto se demostró en los tres entornos, donde las tareas complejas de patrulla y recolección se validaron con éxito en un único ciclo automatizado, resol-

viendo problemas de dependencias de forma transparente al usuario. Sin embargo, la falta de introducir un JSON creado a mano con las dependencias, choca con la idea de la automatización del proceso (lo cual es el principal motivo por el que no es obligatorio su uso).

5.3. Discusión de las hipótesis de las pruebas de usuario

La cantidad de usuarios ($N = 3$) hace que las respuestas no tengan poder de muestra, por lo que es necesario un análisis cualitativo de los datos para poder concluir definitivamente los resultados. Para las hipótesis observamos lo siguiente:

- **H1 (Reducción de tiempos):** Confirmada. Los resultados de la comparativa de tiempos de generación demuestran un ahorro de tiempo sustancial para casi todas las tareas y usuarios. El promedio para completar el segundo árbol de comportamiento se redujo de 221.7 s a solo 71.3 s con la herramienta. No obstante, la excepción del Usuario 2 en el primer árbol (donde tardó 486 s con la herramienta frente a 315 s de forma manual) evidencia que cuando el modelo genera nodos extra o asigna erróneamente variables, el esfuerzo de depuración manual del JSON resultante puede dilatar los tiempos en usuarios que ya son sumamente rápidos de forma manual.
- **H2 (Similitud de ejecución):** Confirmada. Aunque la calidad formal de los árboles se catalogó como inferior (80 % de media de conformidad), el comportamiento observable del NPC cumplió rigurosamente con las expectativas de los usuarios (80 % de conformidad). La inferioridad cualitativa radica en un estilo de programación menos limpio por parte del LLM (inserción de nodos redundantes), pero el resultado final fue funcional y similar al humano.
- **H3 (Buen punto de partida y disminución de carga):** Confirmada. Las altas valoraciones en la utilidad como punto de partida (93.4 %) y la extrema facilidad para modificar la estructura del JSON generado (86.6 %) se traducen en una percepción unánime de que la herramienta efectivamente alivia la carga de trabajo del diseñador (93.4 %). El diseñador puede realizar ajustes superficiales sobre una estructura funcional previa.
- **H4 (Usabilidad y accesibilidad):** Confirmada. El sobresaliente SUS promedio de 90.83 sobre 100 valida que la integración interna del agente dentro de Unreal Engine 5 a través del componente “BTGenerator” resulta sumamente intuitiva y accesible.

5.4. Limitaciones

Este trabajo ha sufrido una serie de limitaciones en cuanto a limitaciones técnicas y a la escasez de usuarios disponibles para las pruebas.

Las limitaciones técnicas se han podido comprobar a la hora de la selección de modelos para generar los árboles de comportamiento. Como bien se ha mencionado anteriormente de los dos modelos, a pesar de cumplir de forma correcta su objetivo, presentan una serie de limitaciones debido a las capacidades computacionales disponibles. El modelo generador de pseudocódigo es un LLM en la nube, teniendo que depender de una API de terceros, lo que no permite un control total del modelo. Adicionalmente, el modelo que asigna las variables de la Blackboard es uno bastante pequeño que puede cometer errores. Además, el sistema RAG se ha construido con árboles de comportamiento de robótica y no de videojuegos y que puede tardar varios segundos en construirse. Otra limitación es la falta de automatización en ciertos aspectos, como en la suscripción de las tareas a la Abstract Factory así como la asignación de su descripción o la generación de las métricas de validación. Aunque los usuarios encuestados encontrasen sencilla la modificación de los árboles generados, es posible que en el futuro busquen grandes cambios en el comportamiento o necesiten reestructurarlo de forma sencilla, lo cual en el estado actual supondría pedirle otra generación distinta a la herramienta en vez de partir con el árbol ya existente.

Finalmente la limitación más grande ha sido la falta de usuarios para poder realizar las pruebas de usuarios, haciendo imposible concluir de forma definitiva las hipótesis planteadas siguiendo los objetivos.

Conclusiones y Trabajo Futuro

El objetivo de este estudio consistía en la creación de una herramienta que, dadas unas tareas y unos condicionales, fuese capaz de generar árboles de comportamiento para personajes no jugador que sirviesen como base para rebajar la carga de trabajo de los diseñadores de videojuegos.

Dados los resultados anteriormente mencionados se han llegado a las siguientes conclusiones.

6.1. Conclusiones

Este trabajo demuestra la viabilidad de generar árboles de comportamiento (BTs) funcionales para NPCs en Unreal Engine 5 a partir de instrucciones en lenguaje natural. Mediante una arquitectura híbrida que combina Mistral Large 3 para la estructura lógica y llama3:8B para la asignación de variables de la Blackboard, el sistema logra mitigar las alucinaciones y traducir con éxito el pseudocódigo generado a formatos JSON legibles.

Para asegurar la calidad del comportamiento, se implementó un sistema de validación automática doble (un autómata NFA para comprobar dependencias y un validador de objetivos finales). Las pruebas experimentales con diseñadores (N = 3) confirmaron la alta usabilidad de la herramienta, alcanzando una excelente puntuación SUS de 90.83/100 y demostrando una reducción drástica de los tiempos de desarrollo frente a la creación manual.

Aunque los árboles automáticos presentan una calidad cualitativa inferior debido a la inserción de algunos nodos redundantes, su ejecución en el juego cumple con lo esperado por los usuarios. La herramienta se consolida como un excelente punto de partida que facilita la edición posterior y disminuye significativamente la carga de trabajo del diseñador. A pesar de que los resultados cuantitativos son exploratorios debido al tamaño de la muestra, el proyecto valida con éxito la integración de LLMs en flujos de producción de videojuegos.

6.2. Trabajo Futuro

Dadas las limitaciones planteadas en la sección 5.4, el trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- Con capacidades computacionales mayores estudiar el uso de modelos locales de mayor tamaño para los modelos usados, así como el uso de técnicas de Fine-tuning para comprobar si su uso alcanzaría o mejoraría los resultados actuales con el fin de no depender de APIs de terceros.
- Mejorar el sistema RAG con árboles de comportamiento de videojuegos y, adicionalmente, que se guarden en disco el almacén vectorial y se puedan buscar por metadatos gracias al uso de Qdrant, suponiendo así una posible mejora en el contexto dado a los modelos y reduciendo la latencia.
- Implementar un agente LLM con una interfaz visual para cambiar los árboles generados de forma más sencilla y que se le pueda dar feedback al generador mediante uso de lenguaje natural.
- Implementar un agente LLM que pueda leer las acciones y descripciones de los proyectos para automatizar su suscripción a la Abstract Factory y su descripción para el paso a la herramienta.
- Implementar un agente LLM que pueda automatizar las pruebas de validación generando el orden de dependencias y una función de objetivos.
- Recopilar más usuarios para las pruebas con el fin de conseguir un número estadísticamente significativo para concluir definitivamente las hipótesis propuestas.

Introduction

Video games are composed of multiple elements that interact with one another to build a coherent and immersive experience for the player, including the player's avatar, environments, interactive objects, and, commonly, non-player characters. These non-player characters (NPCs) play a significant role in the construction of virtual worlds, as they are responsible for providing interactions, responding to player actions, and developing behaviors that are coherent with the context of the game.

The development of these behaviors is a task that involves both design and implementation aspects. Typically, designers determine the actions, routines, and interactions that an NPC must perform, while programmers are responsible for translating these specifications into an executable implementation. For this purpose, different artificial intelligence techniques can be employed, most notably finite state machines (FSMs) and behavior trees (BTs), the latter being the primary object of study of this work.

Furthermore, developing NPC behaviors is an inherently iterative process. During implementation, it is necessary to verify the resulting behavior, detect unforeseen situations, and modify the logic to adapt it to the game's requirements. Consequently, designers and programmers may find themselves forced to successively repeat the phases of behavior design, implementation, evaluation, and modification. In the case of behavior trees, these changes frequently involve creating, reorganizing, or modifying nodes, conditions, and execution relationships, which can increase the effort required as behavior complexity grows.

In addition to this development cost, there is a requirement for specific technical knowledge to correctly build and modify these structures. Using a system based on behavior trees requires understanding both their execution logic and the elements that compose them and their relationships, which can hinder their use by developers with less experience in artificial intelligence or by profiles not specialized in programming.

In this context, the automatic generation of behavior trees from natural language descriptions constitutes a possible avenue to reduce the effort associated with their creation and modification. Allowing a user to describe the desired behavior through a natural language specification and automatically obtain an executable structure could reduce part of the manual work required during development, facilitate iteration over behaviors, and lower the technical barriers associated with their

implementation.

The emergence of Large Language Models (LLMs), which are capable of interpreting natural language instructions and generating complex structures from them, opens up a new possibility to automate this process. For this reason, this work studies the use of language models for the automatic generation of behavior trees aimed at controlling non-player characters, analyzing the possibilities and limitations of this approach in the context of video game development.

6.3. Motivation

The growing complexity of video games has also increased the complexity of the behaviors that their non-player characters must implement. As a greater number of actions, situations, and interaction possibilities are incorporated, the structures responsible for controlling these behaviors require a larger number of elements and relationships between them. In this context, behavior trees present characteristics such as modularity and scalability that are especially suitable for representing complex behaviors ([Lim et al. \(2010\)](#)).

However, these characteristics do not eliminate the cost associated with the creation and maintenance of the trees. The manual construction of behaviors can be laborious and require specialized knowledge, especially when behaviors must be modified or expanded iteratively. The literature precisely points out the effort involved for designers in developing NPC behaviors and proposes mechanisms aimed at automating their expansion ([Florez-Puga et al. \(2008\)](#)). Likewise, there are articles that highlight the need to facilitate the interaction between the design aspects and the technical aspects of behavior implementation ([Llansó et al. \(2009\)](#)).

The difficulty of authoring behavior trees has also motivated research into tools designed to facilitate their creation. Some authors propose an authoring tool based on a visual interface that seeks to simplify the construction of these trees and bring their editing closer to the design process ([Becroft et al. \(2011\)](#)). These works show that reducing the effort required to specify and modify behaviors is a problem that predates the emergence of LLMs.

Consequently, different approaches have been developed to fully or partially automate the construction of behavior trees. Among them are methods based on the reuse of behaviors, automatic evolution techniques, and synthesis methods from formal specifications, demonstrating that the generation of behavior trees is a consolidated line of research. These approaches allow reducing part of the manual work, although they generally require structured representations of the problem, evaluation functions, demonstrations, or domain-specific knowledge.

The appearance of LLMs introduces a new possibility to address this problem, as these models can process instructions expressed in natural language and generate structures from them. Instead of requiring the user to directly define the internal structure of the behavior tree, it can be proposed that they describe the desired behavior using natural language, and the system handles the translation to a structured representation. This approach allows exploring an interaction closer to the design process and can reduce the technical barriers associated with authoring be-

haviors. In this context, this work is situated at the intersection between research on the automatic generation of behavior trees and recent advances in LLMs.

6.4. Objectives

The general objective of this project is the implementation of a tool that can be used in a simple and intuitive manner by designers, both experts and those without a technical background, to generate behavior trees more efficiently than they could manually, and with a quality similar to that of human-created trees, in order to alleviate their workload. The aim is not to replace the designer's work, but rather to provide them with a useful foundation that yields easily modifiable structures to refine certain aspects or easily alter the generated behaviors.

To meet these objectives, the following goals have been proposed during this work:

- Implementation of a library of tasks and conditionals necessary to generate different behaviors.
- Design and implementation of various scenarios in which the sought behaviors can be tested in real-time and visually.
- Comparison of different implementation methods and language models (execution times and generation quality) to choose the ideal one to fulfill the general objective.
- Implementation of the generating tool using different AI techniques that achieve optimal generation.
- Implementation of a validation system for the generated trees to verify that they are valid in terms of respecting node dependencies and fulfilling objectives.
- Execution of a testing plan to validate the fulfillment of the general objectives.

6.5. Work Plan

The work plan consists of several steps:

1. **November - February:** Development and implementation of several scenarios in a game engine containing what is necessary for generation: a library of tasks and conditionals, the ability to execute the generated BTs, and a way to communicate with the generation tool.
2. **February - April:** Implementation of a tool that, through large language models, is capable of generating BTs from natural language that can satisfy a desired behavior.

3. **April - June:** Development of an automatic validation system for the generated BTs.
4. **July - August:** Design and subsequent execution of tests with target users to measure the correct fulfillment of the objectives.

6.6. Methodology and Technology

To develop this project, various tools commonly used in video game development have been used, keeping the target audience in mind.

- **Unreal Engine 5.7:** Unreal Engine is one of the industry's leading engines, characterized by its technical robustness. It also features a native event-driven behavior tree system and a Blackboard component, which is necessary for the non-player character's interaction with the environment. This engine was selected due to its widespread use in the industry, making it easier for the designers at whom the tool is aimed to work in a familiar environment. Within this engine, a library of tasks and conditionals necessary for behavior creation was established, along with several scenarios to test the behavior tree generations.
- **Python 3:** This is a high-level programming language that is widely adopted due to its versatility and its standard position in artificial intelligence development. Since it constitutes the backend of the project, and in line with the ease-of-use objectives of this tool, the section developed in this language is completely opaque to the user. The tool developed in Python is responsible for the entire generation of the trees, leveraging available LLM orchestration modules, with LangChain being the one used in this work.
- **Hugging Face:** This is an open-source ecosystem for the distribution of machine learning models, libraries, and natural language processing datasets. In this work, it was used to build a RAG system, serving as the source repository from which the data was retrieved, and providing the embedding model.
- **Mistral AI:** This is a generative artificial intelligence company that offers large language models. One of the models used throughout this work was Mistral Large 3 via the company's API. This model has 675B total parameters and 41B active parameters, and is designed as a 'Mixture of Experts' system, making it a robust model that can fulfill the tool's purpose.
- **Ollama:** This is an open-source tool designed to manage and run language models locally, ensuring low latency and eliminating API call costs. This environment hosts and runs locally another model used in this work, the llama3:8B model, which features 8B parameters. Although it is a lighter model and is not as precise as the previously mentioned one, its use proved to be effective in this work

6.7. Document Structure

This document is structured in accordance with the previously mentioned work plan:

1. **Chapter 2: State of Art.** This chapter presents the theoretical and technological framework surrounding this work. It reviews different behavior generation methods used in video games, detailing their definitions and evolution in both industry and academia, with a special emphasis on behavior trees. It also recounts the evolution and current context of language models to ultimately contextualize the project's research line of automatic behavior generation.
2. **Chapter 3: Behavior Tree generation tool.** This chapter describes the development of the tool implemented in this work. It is divided into four sections:
 - a) **Pipeline design of the proposed behavior tree generation model:** Describes the workflow design of the tool conceived to satisfy the proposed objectives.
 - b) **Description of the environment in a game engine:** Details the environment created in a game engine to implement various tasks and conditionals necessary to generate different types of behaviors, as well as the development of several scenarios where the behavior trees can be tested.
 - c) **Behavior tree generation tool:** Explains the implementation of the tool that generates a JSON file with a behavior tree from the description of the desired behavior in natural language.
 - d) **Automatic validation of behavior trees:** Describes the methods designed to validate behavior trees, specifically the validation of task dependency fulfillment and goal fulfillment. It also describes several tests conducted to verify the system's validity.
3. **Chapter 4: Evaluation and results.** This chapter recounts the user evaluation tests designed and executed to validate a series of hypotheses, and reviews the obtained results.
4. **Chapter 5: Discussion.** This chapter discusses the implementation and results of all sections of the project, and comments on the different limitations encountered during its development.
5. **Chapter 6: Conclusions.** This chapter details the conclusions drawn from the project's results and proposes future work in light of the limitations.

6.8. Link to Repository

The source code of this work is available in a public GitHub repository. The repository is structured as follows:

- **Análisis:** Contains the CSV of user test responses and the Jupyter Notebook with the code to generate the charts (see 4.2).
- **bt-dataset-parser:** Python project in which the dataset needed to demonstrate the RAG system is downloaded and processed (see 3.3).
- **BTGenerator:** An Unreal project that served as the base for the environments (see 3.2).
- **llm-bt-generator:** A Python project containing the BT generator tool (see 3.3).
- **Memoria:** The LaTeX source code of this document.

The link to the repository is: <https://github.com/pablosm196/TFM>.

6.9. Use of Artificial Intelligence

This document has been written with the selective support of generative Artificial Intelligence tools for drafting tasks. Their sole use consisted of consolidating the ideas originally established in the document by reformulating certain paragraphs, which were subsequently reviewed and edited. In any case, the structure of the document, as well as the ideas it transmits and the final responsibility for the content, lies with the author.

Conclusions and Future Work

The objective of this study was to create a tool that, given a set of tasks and conditions, would be capable of generating behavior trees for non-player characters (NPCs) to serve as a foundation to reduce the workload of video game designers.

Based on the aforementioned results, the following conclusions have been reached.

6.10. Conclusions

This work demonstrates the viability of generating functional behavior trees (BTs) for NPCs in Unreal Engine 5 from natural language instructions. Through a hybrid architecture that combines Mistral Large 3 for the logical structure and llama3:8B for Blackboard variable assignment, the system successfully mitigates hallucinations and translates the generated pseudocode into readable JSON formats.

To ensure behavior quality, a double automatic validation system was implemented (a nondeterministic finite automaton (NFA) to check dependencies and a final objective validator). Experimental tests with designers ($N = 3$) confirmed the tool's high usability, achieving an excellent System Usability Scale (SUS) score of 90.83/100 and demonstrating a drastic reduction in development times compared to manual creation.

Although the automatically generated trees exhibit lower qualitative quality due to the inclusion of some redundant nodes, their execution in the game meets user expectations. The tool establishes itself as an excellent starting point that facilitates subsequent editing and significantly decreases the designer's workload. Despite the quantitative results being exploratory due to the sample size, the project successfully validates the integration of LLMs into game production workflows.

6.11. Future Work

Given the limitations presented in section 5.4, the future work of this study can focus on several efforts:

- With greater computational capabilities, study the use of larger local models for the tasks performed, as well as applying fine-tuning techniques to verify if

their use would match or improve current results in order to avoid dependency on third-party APIs.

- Improve the RAG system by incorporating video game BTs, saving the vector store on disk, and enabling metadata searches using Qdrant, which could potentially improve the context provided to the models and reduce latency.
- Implement an LLM agent with a visual interface to modify the generated BTs more easily, allowing natural language feedback to be provided to the generator.
- Implement an LLM agent capable of reading the project's actions and descriptions to automate their subscription to the Abstract Factory and generate descriptions for the tool.
- Implement an LLM agent to automate the validation tests by generating the dependency order and a goal function.
- Gather more users for the trials in order to achieve a statistically significant sample to definitively conclude the proposed hypotheses.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

- AGIS, R. A., GOTTIFREDI, S. y GARCÍA, A. J. An event-driven behavior trees extension to facilitate non-player multi-agent coordination in video games. *Expert Systems with Applications*, vol. 155, página 113457, 2020. ISSN 0957-4174.
- AHN, M., BROHAN, A., BROWN, N., CHEBOTAR, Y., CORTES, O., DAVID, B., FINN, C., FU, C., GOPALAKRISHNAN, K., HAUSMAN, K., HERZOG, A., HO, D., HSU, J., IBARZ, J., ICHTER, B., IRPAN, A., JANG, E., RUANO, R. J., JEFFREY, K., JESMONTH, S., JOSHI, N. J., JULIAN, R., KALASHNIKOV, D., KUANG, Y., LEE, K.-H., LEVINE, S., LU, Y., LUU, L., PARADA, C., PASTOR, P., QUIAMBAO, J., RAO, K., RETTINGHOUSE, J., REYES, D., SERMANET, P., SIEVERS, N., TAN, C., TOSHEV, A., VANHOUCKE, V., XIA, F., XIAO, T., XU, P., XU, S., YAN, M. y ZENG, A. Do as i can, not as i say: Grounding language in robotic affordances. 2022.
- AO, J., WU, F., WU, Y., SWIKIR, A. y HADDADIN, S. Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning. 2025.
- BANGOR, A., KORTUM, P. T. y MILLER, J. T. An empirical evaluation of the system usability scale. *International Journal of Human-Computer Interaction*, vol. 24(6), páginas 574–594, 2008.
- BECROFT, D., BASSETT, J., MEJIA, A., RICH, C. y SIDNER, C. AIPaint: A sketch-based behavior tree authoring tool. *Proc. AAAI Artif. Intell. Interact. Digit. Enterain. Conf.*, vol. 7(1), páginas 2–7, 2011.

- BEN-ARI, M. y MONDADA, F. *Finite State Machines*, páginas 55–61. Springer International Publishing, Cham, 2018. ISBN 978-3-319-62533-1.
- BENGIO, Y., DUCHARME, R., VINCENT, P. y JANVIN, C. A neural probabilistic language model. *J. Mach. Learn. Res.*, vol. 3(null), página 1137–1155, 2003. ISSN 1532-4435.
- BERGER, A. L., DELLA PIETRA, S. A. y DELLA PIETRA, V. J. A maximum entropy approach to natural language processing. *Computational Linguistics*, vol. 22(1), páginas 39–71, 1996.
- BROOKE, J. SUS-A quick and dirty usability scale. *Usability evaluation in industry*, vol. 189(194), 1996.
- BROWN, T. B., MANN, B., RYDER, N., SUBBIAH, M., KAPLAN, J., DHARIWAL, P., NEELAKANTAN, A., SHYAM, P., SASTRY, G., ASKELL, A., AGARWAL, S., HERBERT-VOSS, A., KRUEGER, G., HENIGHAN, T., CHILD, R., RAMESH, A., ZIEGLER, D. M., WU, J., WINTER, C., HESSE, C., CHEN, M., SIGLER, E., LITWIN, M., GRAY, S., CHESSE, B., CLARK, J., BERNER, C., MCCANDLISH, S., RADFORD, A., SUTSKEVER, I. y AMODEI, D. Language models are few-shot learners. 2020.
- CAMACHO, A., TRIANTAFILLOU, E., MUISE, C., BAIER, J. y MCILRAITH, S. Non-deterministic planning with temporally extended goals: LTL over finite and infinite traces. *Proc. Conf. AAAI Artif. Intell.*, vol. 31(1), 2017.
- CHO, K., VAN MERRIËNBOER, B., GULCEHRE, C., BAHDANAU, D., BOUGARES, F., SCHWENK, H. y BENGIO, Y. Learning phrase representations using RNN encoder–decoder for statistical machine translation. En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (editado por A. Moschitti, B. Pang y W. Daelemans), páginas 1724–1734. Association for Computational Linguistics, Doha, Qatar, 2014.
- CHOI, J.-W., KIM, H., ONG, H., YOON, Y., JANG, M., KIM, D. y KIM, J. Reac-tree: Hierarchical llm agent trees with control flow for long-horizon task planning. 2026.
- COLLEDANCHISE, M., ALMEIDA, D. y OGREN, P. Towards blended reactive planning and acting using behavior trees. En *2019 International Conference on Robotics and Automation (ICRA)*, página 8839–8845. IEEE, 2019.
- COLLEDANCHISE, M. y OGREN, P. How behavior trees modularize hybrid control systems and generalize sequential behavior compositions, the subsumption architecture, and decision trees. *IEEE Transactions on Robotics*, vol. 33, páginas 372–389, 2016.
- DEVLIN, J., CHANG, M.-W., LEE, K. y TOUTANOVA, K. Bert: Pre-training of deep bidirectional transformers for language understanding. 2019.

- ELMAN, J. L. Finding structure in time. *Cognitive Science*, vol. 14(2), páginas 179–211, 1990. ISSN 0364-0213.
- FATHONI, K., HAKKUN, R. Y. y NURHADI, H. A. T. Finite state machines for building believable non-playable character in the game of khalid ibn al-walid. *Journal of Physics: Conference Series*, vol. 1577(1), página 012018, 2020.
- FAUZI, R., HARIADI, M., NUGROHO, S. y LUBIS, M. Defense behavior of real time strategy games: Comparison between hfsm and fsm. *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 13(2), páginas 634–642, 2019. ISSN 2502-4752. Publisher Copyright: © 2019 Institute of Advanced Engineering and Science. All rights reserved.
- FLOREZ-PUGA, G., GÓMEZ-MARTÍN, M. y DÍAZ AGUDO, B. Dynamic expansion of behaviour trees. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 4, páginas 36–41, 2008.
- GAJARDO, I., BESOAIN, F. y BARRIGA, N. A. Introduction to behavior algorithms for fighting games. En *2019 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON)*, página 1–6. IEEE, 2019.
- GANAPATHI, A., SIVAKUMAR, P., ELANGO, A., GUPTA, H. y PANDA, N. Exploring npc behaviors in games through finite automata. En *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*, páginas 1–8. 2024.
- GUGLIERMO, S., CÁCERES DOMÍNGUEZ, D., IANNOTTA, M., STOYANOV, T. y SCHAFFERNICHT, E. Evaluating behavior trees. *Robotics and Autonomous Systems*, vol. 178, página 104714, 2024. ISSN 0921-8890.
- GUO, S. y HOU, S. Analyzing Approaches to Extending and Implementing Behavior Trees in Computer Games. *Applied and Computational Engineering*, vol. 199(1), páginas 139–153, 2025.
- HOCHREITER, S. y SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, vol. 9(8), páginas 1735–1780, 1997. ISSN 0899-7667.
- HOFFMEISTER, L. M., SCASSELLATI, B. y RAKITA, D. Towards zero-knowledge task planning via a language-based approach. 2026.
- HOU, X., ZHAO, Y., WANG, S. y WANG, H. Model context protocol (mcp): Landscape, security threats, and future research directions. *ACM Trans. Softw. Eng. Methodol.*, 2026. ISSN 1049-331X. Just Accepted.
- HU, W., ZHANG, Q. y MAO, Y. Component-based hierarchical state machine — a reusable and flexible game ai technology. 2011.
- HUANG, W., XIA, F., XIAO, T., CHAN, H., LIANG, J., FLORENCE, P., ZENG, A., TOMPSON, J., MORDATCH, I., CHEBOTAR, Y., SERMANET, P., BROWN, N., JACKSON, T., LUU, L., LEVINE, S., HAUSMAN, K. y ICHTER, B. Inner monologue: Embodied reasoning through planning with language models. 2022.

- HYZY, M., BOND, R., MULVENNA, M., BAI, L., DIX, A., LEIGH, S. y HUNT, S. System Usability Scale Benchmarking for Digital Health Apps: Meta-analysis. *JMIR mHealth and uHealth*, vol. 10(8), página e37290, 2022.
- IOVINO, M., FÖRSTER, J., FALCO, P., CHUNG, J. J., SIEGWART, R. y SMITH, C. Comparison between behavior trees and finite state machines. *IEEE Transactions on Automation Science and Engineering*, vol. 22, páginas 21098–21117, 2025.
- IOVINO, M., SCUKINS, E., STYRUD, J., ÖGREN, P. y SMITH, C. A survey of behavior trees in robotics and ai. 2020a.
- IOVINO, M., SCUKINS, E., STYRUD, J., ÖGREN, P. y SMITH, C. A survey of behavior trees in robotics and ai. *Robotics and Autonomous Systems*, vol. 154, página 104096, 2022. ISSN 0921-8890.
- IOVINO, M., STYRUD, J., FALCO, P. y SMITH, C. Learning behavior trees with genetic programming in unpredictable environments. 2020b.
- IOVINO, M., STYRUD, J., FALCO, P. y SMITH, C. A framework for learning behavior trees in collaborative robotic applications. 2023.
- ITO, R., JIMBO, N. y IHARA, K. Pilot study on llm-based behavior tree generation. *Proceedings of the Annual Conference of JSAI*, vol. JSAI2026, páginas 2G1OS2103–2G1OS2103, 2026.
- IZZO, R. A., BARDARO, G. y MATTEUCCI, M. Btgenbot: Behavior tree generation for robotic tasks with lightweight llms. En *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, página 9684–9690. IEEE, 2024.
- JELINEK, F. y MERCER, R. L. Interpolated estimation of Markov source parameters from sparse data. En *Proc. Workshop on Pattern Recognition in Practice*. 1980.
- KIMDONGJU y KOHSEOK-JOO. Intelligent npc ai based on fsm and reinforcement learning: Implementation study of a 2d mobile idle rpg game. *The Transactions of the Korea Information Processing Society*, vol. 14(6), páginas 480–488, 2025.
- LEE, M. An artificial intelligence evaluation on fsm-based game npc. *Journal of Korea Game Society*, vol. 14, páginas 127–135, 2014.
- LI, F., WANG, X., LI, B., WU, Y., WANG, Y. y YI, X. A study on training and developing large language models for behavior tree generation. 2024.
- LIM, C., BAUMGARTEN, R. y COLTON, S. Evolving behaviour trees for the commercial game defcon. En *Applications of Evolutionary Computation - EvoApplications 2010*, número PART 1 en Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), páginas 100–110. 2010. ISBN 3642122388. European Workshop on Bio-inspired Algorithms in Games 2010, EvoGAMES 2010 ; Conference date: 07-04-2010 Through 09-04-2010.

- LIU, B., JIANG, Y., ZHANG, X., LIU, Q., ZHANG, S., BISWAS, J. y STONE, P. Llm+p: Empowering large language models with optimal planning proficiency. 2023.
- LLANSÓ, D., GÓMEZ-MARTÍN, M. y GONZÁLEZ-CALERO, P. Self-validated behaviour trees through reflective components. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 5, páginas 70–75, 2009.
- LYKOV, A. y TSETSERUKOU, D. Llm-brain: Ai-driven fast generation of robot behaviour tree based on large language model. 2023.
- MERINO-FIDALGO, S., SÁNCHEZ-GIRÓN, C., ZALAMA, E., GÓMEZ-GARCÍA-BERMEJO, J. y DUQUE-DOMINGO, J. Behavior tree generation and adaptation for a social robot control with llms. *Robotics and Autonomous Systems*, vol. 194, página 105165, 2025. ISSN 0921-8890.
- MIKOLOV, T., CHEN, K., CORRADO, G. y DEAN, J. Efficient estimation of word representations in vector space. 2013.
- NUR FITRIANI DEWI, E., RACHMAN, A. y ZAHIR, H. Implementation of hierarchical finite state machine for controlling 2d character animation in action video game. páginas 1–6. 2023.
- PADURARU, C., STEFANESCU, A. y PADURARU, M. Enhancing game ai behaviors with large language models and agentic ai. 2025.
- PARK, J. S., O'BRIEN, J. C., CAI, C. J., MORRIS, M. R., LIANG, P. y BERNSTEIN, M. S. Generative agents: Interactive simulacra of human behavior. 2023.
- PARK, S., DURFEE, E. H. y BIRMINGHAM, W. P. Use of markov chains to design an agent bidding strategy for continuous double auctions. *J. Artif. Intell. Res.*, vol. 22, páginas 175–214, 2004.
- PARTLAN, N., SOTO, L., HOWE, J., SHRIVASTAVA, S., EL-NASR, M. S. y MARSELLA, S. Evolvingbehavior: Towards co-creative evolution of behavior trees for game npcs. 2022.
- PERES, S. C., PHAM, T. y PHILLIPS, R. Validation of the system usability scale (sus): Sus in the wild. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 57(1), páginas 192–196, 2013.
- PETERS, M. E., NEUMANN, M., IYYER, M., GARDNER, M., CLARK, C., LEE, K. y ZETTLEMOYER, L. Deep contextualized word representations. En *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)* (editado por M. Walker, H. Ji y A. Stent), páginas 2227–2237. Association for Computational Linguistics, New Orleans, Louisiana, 2018.
- RADFORD, A. y NARASIMHAN, K. Improving language understanding by generative pre-training. 2018.

- SAGREDO-OLIVENZA, I., GÓMEZ-MARTÍN, P. y GÓMEZ-MARTÍN, M. Trained behavior trees: Programming by demonstration to support ai game designers. *IEEE Transactions on Games*, vol. 11, páginas 5–14, 2017.
- SCHICK, T., DWIVEDI-YU, J., DESSÌ, R., RAILEANU, R., LOMELI, M., ZETTLEMOYER, L., CANCEDDA, N. y SCIALOM, T. Toolformer: Language models can teach themselves to use tools. 2023.
- SCHULHOFF, S., ILIE, M., BALEPUR, N., KAHADZE, K., LIU, A., SI, C., LI, Y., GUPTA, A., HAN, H., SCHULHOFF, S., DULEPET, P. S., VIDYADHARA, S., KI, D., AGRAWAL, S., PHAM, C., KROIZ, G., LI, F., TAO, H., SRIVASTAVA, A., COSTA, H. D., GUPTA, S., ROGERS, M. L., GONCEARENCO, I., SARLI, G., GALYNKER, I., PESKOFF, D., CARPUAT, M., WHITE, J., ANADKAT, S., HOYLE, A. y RESNIK, P. The prompt report: A systematic survey of prompt engineering techniques. 2025.
- SEKHAVAT, Y. Behavior trees for computer games. *International Journal on Artificial Intelligence Tools*, vol. 26, página 1730001, 2017.
- SONG, C. H., WU, J., WASHINGTON, C., SADLER, B. M., CHAO, W.-L. y SU, Y. Llm-planner: Few-shot grounded planning for embodied agents with large language models. 2023.
- THOMPSON, T. Cyber demons. <https://www.gamedeveloper.com/design/cyber-demons-the-ai-of-doom-2016->, 2018. Accessed: 2026-8-27.
- TREMBLAY, H., PONTON, D., GONZALEZ, L., FRANCILLETTE, Y. y BOUCHARD, B. Breaking down the challenge: A comprehensive framework for evaluating enemy difficulty with automated behavior tree analytics. *Entertainment Computing*, vol. 57, página 101096, 2026. ISSN 1875-9521.
- ULUDAGLI, C. y OGUZ, K. Non-player character decision-making in computer games. *Artificial Intelligence Review*, vol. 56, páginas 1–33, 2023.
- VALMEEKAM, K., MARQUEZ, M., OLMO, A., SREEDHARAN, S. y KAMBHAMPATI, S. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. 2023.
- VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L. y POLOSUKHIN, I. Attention is all you need. 2023.
- WANG, L., MA, C., FENG, X., ZHANG, Z., YANG, H., ZHANG, J., CHEN, Z., TANG, J., CHEN, X., LIN, Y., ZHAO, W. X., WEI, Z. y WEN, J. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, vol. 18(6), 2024. ISSN 2095-2236.
- WEI, J., BOSMA, M., ZHAO, V. Y., GUU, K., YU, A. W., LESTER, B., DU, N., DAI, A. M. y LE, Q. V. Finetuned language models are zero-shot learners. 2022.

- WEI, J., WANG, X., SCHUURMANS, D., BOSMA, M., ICHTER, B., XIA, F., CHI, E., LE, Q. y ZHOU, D. Chain-of-thought prompting elicits reasoning in large language models. 2023.
- RONG WEN, C. Q. S. D. X. W. H. C. S. W. D. Y. J. X. J. Tool learning with large language models: a survey. *Frontiers of Computer Science*, vol. 19, páginas 198343–, 2025. ISSN 2095-2228.
- YANNAKAKIS, M. *Hierarchical State Machines*, vol. 1872, páginas 315–330. 2001. ISBN 978-3-540-67823-6.
- YAO, S., ZHAO, J., YU, D., DU, N., SHAFRAN, I., NARASIMHAN, K. y CAO, Y. React: Synergizing reasoning and acting in language models. 2023.
- ZHOU, D., SCHÄRLI, N., HOU, L., WEI, J., SCALES, N., WANG, X., SCHUURMANS, D., CUI, C., BOUSQUET, O., LE, Q. y CHI, E. Least-to-most prompting enables complex reasoning in large language models. 2023.
- ÖGREN, P. y SPRAGUE, C. I. Behavior trees in robot control systems. *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5(1), página 81–107, 2022. ISSN 2573-5144.

JSON Schema para los árboles de comportamiento generados

Listing A.1: JSON Schema (Draft 2020-12) de los árboles de comportamiento generados

```

1  {
2    "$schema": "https://json-schema.org/draft/2020-12/schema",
3    "$id": "https://example.com/behaviour-tree.schema.json",
4    "title": "Behaviour Tree",
5    "type": "object",
6    "required": ["Blackboard", "Node"],
7    "additionalProperties": false,
8
9    "properties": {
10     "Blackboard": {
11       "type": "array",
12       "description": "Blackboard",
13       "items": {
14         "type": "object",
15         "additionalProperties": {
16           "type": "string",
17           "enum": ["Bool", "Int", "Float", "Vector",
18                  ↪ "Object", "String"]
19         }
20       }
21     },
22     "Node": {
23       "$ref": "#/$defs/TreeNodeWrapper"
24     }
25   },
26
27   "$defs": {
28     "TreeNodeWrapper": {
29       "type": "object",

```

```

30     "description": "Nodo del arbol",
31     "minProperties": 1,
32     "maxProperties": 1,
33     "additionalProperties": {
34         "$ref": "#/$defs/TreeNode"
35     }
36 },
37
38 "TreeNode": {
39     "type": "object",
40     "required": ["Type", "Decorators", "Nodes"],
41     "additionalProperties": false,
42
43     "properties": {
44         "Type": {
45             "type": "string",
46             "enum": ["Selector", "Sequence", "Task", "Parallel"]
47         },
48
49         "Name": {
50             "type": "string",
51             "description": "Nombre opcional del nodo"
52         },
53
54         "Decorators": {
55             "type": "array",
56             "items": {
57                 "type": "object",
58                 "additionalProperties": {
59                     "type": "string"
60                 }
61             }
62         },
63
64         "Task": {
65             "type": "string",
66             "description": "Nombre de la tarea"
67         },
68
69         "BlackboardEntries": {
70             "type": "array",
71             "description": "Entradas del Blackboard usadas por
72                 ⇨ el nodo",
73             "items": {
74                 "type": "string"
75             }
76         },
77
78         "Nodes": {

```

```
78         "type": "array",
79         "description": "Nodos hijos",
80         "items": {
81             "$ref": "#/$defs/TreeNodeWrapper"
82         }
83     }
84 }
85 ,
86 "Nodes": {
87     "type": "array",
88     "description": "Nodos hijos",
89     "minItems": 0,
90     "items": {
91         "type" : "object",
92         "required": ["Node"],
93         "additionalProperties" : false,
94         "properties": {
95             "Node": {
96                 "$ref": "#/$defs/TreeNode"
97             }
98         }
99     }
100 },
101
102 "allOf": [
103     {
104         "if": {
105             "properties": { "Type": { "const": "Task" } }
106         },
107         "then": {
108             "required": ["Task"]
109         }
110     }
111 ]
112 }
113 }
114 }
```


Apéndice B

Prompt del sistema para el LLM generador del pseudocódigo

You are an expert in behavior trees and algorithm design.

Your task is to convert a high-level task description into a structured pseudocode

Input:

- * A task description

Requirements:

- * Use ONLY the provided actions and ONLY the provided conditions. Do NOT invent
- * Produce a deterministic and unambiguous pseudocode.
- * The structure must be easy to parse programmatically.
- * Explicitly represent:
 - Sequence (ordered execution)
 - Selector (fallback / OR logic / conditionals)
 - Conditions (checks)
 - Actions (leaf nodes)
- * Avoid natural language ambiguity. Use a strict format.

Pseudocode Format Rules:

- * Use uppercase keywords: SEQUENCE, SELECTOR, IF, THEN, ELSE, END
- * You don't have to use all the keywords, only use the necessary ones.
- * Indentation must reflect hierarchy
- * Each condition must be written as: IF condition_name. Use ONLY the names from
- * Each action must be written as: DO action_name. Use ONLY the names from the "
- * ONLY blocks of SEQUENCE and SELECTOR must always be explicitly closed with EN
- * No free text explanations, only pseudocode

Figura B.1: Prompt del sistema para el LLM generador del pseudocódigo (primera parte)

Behavior Tree Mapping:

- * SEQUENCE: executes children in order until one fails. Perfect for when action
- * SELECTOR: executes children until one succeeds. Perfect for when you have to
- * IF condition THEN ... ELSE ... : conditional branching

Reasoning Procedure:

Before generating the pseudocode, internally perform the following reasoning.

Do NOT output these steps.

Phase 1

Understand the global objective.

Identify:

- * the final goal;
- * mandatory subtasks;
- * optional subtasks.

Phase 2

Analyse the available actions.

For every action determine internally:

- * what it achieves;
- * when it is useful;
- * whether another action already achieves the same purpose.

Discard unnecessary actions.

Phase 3

Analyse the available conditions.

Determine which conditions are actually necessary for solving the task.

Ignore irrelevant conditions.

Figura B.2: Prompt del sistema para el LLM generador del pseudocódigo (segunda parte)

Phase 4

Construct the execution plan.

Prefer:

- * the minimum number of actions;
- * the minimum tree depth;
- * deterministic execution;
- * reusable sequences.

Only introduce a Selector when alternative behaviours are required.

Only introduce an IF when behaviour explicitly depends on a condition.

Phase 5

Verify the solution.

Internally verify that:

- * every action belongs to the available actions;
- * every condition belongs to the available conditions;
- * every IF has a THEN;
- * every SEQUENCE and SELECTOR has a matching END;
- * no END closes an IF or an ELSE;
- * the algorithm solves the requested task;
- * there are no redundant actions;
- * there are no unreachable branches.

Only after all checks succeed, generate the final pseudocode.

Do NOT output your reasoning.

Output:

Return ONLY the pseudocode, no explanations.

Figura B.3: Prompt del sistema para el LLM generador del pseudocódigo (tercera parte)

Example Input:

Task: "Open a locked door"

Actions: [find_key, move_to_door, open_door]

Conditions: [has_key, door_is_locked]

Example Output:

'''

SEQUENCE

 IF has_key THEN

 DO move_to_door

 DO open_door

 ELSE

 SEQUENCE

 DO find_key

 DO move_to_door

 DO open_door

 END

END

'''

The following examples are not documentation.

They are demonstrations of valid planning patterns.

Before generating the solution:

1. Find the example that is most similar to the current task.
2. Reuse its planning structure whenever possible.
3. Adapt only the actions and conditions that differ.
4. Preserve the overall control flow unless the task requires a different one.

Do not copy examples blindly.

Adapt them to the current task.

The "prompt" field is the user prompt, and the "code" field is the pseudocode generated.

{{ejemplos sacados del RAG}}

List of actions ("actions" list): {{(lista de tareas, tanto las que requieren de una acción como las que no)}}.

List of conditions ("conditions" list) : {{(lista de condicionales)}}.

Figura B.4: Prompt del sistema para el LLM generador del pseudocódigo (cuarta parte)

JSON de dependencias de tareas en un árbol de comportamiento

Listing C.1: Ejemplo del formato que deben seguir los JSON con el orden de dependencias de tareas

```

1 {
2   {
3     "Tasks" : [
4       {
5         "Name" : "HasLineOfSight?",
6         "Childs" : [
7           {
8             "Name" : "ChasePlayer",
9             "Childs" : [
10              {
11                "Name" : "MoveTo",
12                "Childs" : [
13                  {
14                    "Name" : "IsItNear?",
15                    "Childs" : [
16                      {
17                        "Name" : "Attack",
18                        "Childs" : []
19                      }
20                    ]
21                  }
22                ]
23              }
24            ]
25          }
26        ]
27      },
28      {
29        "Name" : "SelectWaypoint",
30        "Childs" : [

```

```
31      {
32          "Name" : "MoveTo",
33          "Childs" : []
34      }
35  ]
36 }
37 ]
38 }
39 }
```

Apéndice	D
----------	----------

Formulario de opiniones de la herramienta

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

Herramienta para la generación de Behavior Trees

* Indica que la pregunta es obligatoria

1. Usuario *

Usabilidad de la herramienta

En esta sección se le realizarán preguntas sobre la usabilidad de la herramienta.

2. Creo que me gustaría utilizar este sistema con frecuencia *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

3. Encontré el sistema innecesariamente complejo *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

4. Pensé que el sistema era fácil de usar *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

5. Creo que necesitaría el apoyo de un técnico para poder utilizar este sistema *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

6. Encontré que las funciones de este sistema estaban bien integradas *

Marca solo un óvalo.

1 2 3 4 5
Totalmente ☐ ☐ ☐ ☐ ☐ Totalmente de acuerdo

Figura D.1: Formulario de recogida de datos demográficos (primera parte)

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

7. Encontré que las funciones de este sistema estaban bien integradas *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

8. Pensé que había demasiada inconsistencia en este sistema *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

9. Me imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

10. Me imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

11. Encontré el sistema muy complicado de usar *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

12. Me sentí muy seguro usando el sistema *

Marca solo un óvalo.

	1	2	3	4	5	
Total	☐	☐	☐	☐	☐	Totalmente de acuerdo

Figura D.2: Formulario de recogida de datos demográficos (segunda parte)

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

13. Necesitaba aprender muchas cosas antes de empezar con este sistema *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

Calidad de la herramienta

14. El tiempo de generación de BTs ha sido adecuado *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

15. La ejecución de los BTs generados ha resultado ser la esperada *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

16. La calidad de los BTs generados es inferior a los generados por un humano *

Marca solo un óvalo.

	1	2	3	4	5
Totale	☐	☐	☐	☐	☐

Totalmente de acuerdo

17. ¿Los BTs generados han introducido más nodos de los necesarios? Contando todos los BTs generados, ¿cuántos?

18. ¿Cómo de distintos son los BTs creados por ti con los BTs generados?

Figura D.3: Formulario de recogida de datos demográficos (tercera parte)

26/8/26, 16:52

Herramienta para la generación de Behavior Trees

19. Los BTs generados son un buen punto de partida a la hora de crear BTs *

Marca solo un óvalo.

	1	2	3	4	5	
Tota	☐	☐	☐	☐	☐	Totalmente de acuerdo

20. ¿Consideras necesaria la intervención humana en esta herramienta para poder refinar los resultados o los BTs generados han sido lo suficientemente buenos como para no tener que modificarlos?

21. Los BTs generados se pueden modificar fácilmente *

Marca solo un óvalo.

	1	2	3	4	5	
Tota	☐	☐	☐	☐	☐	Totalmente de acuerdo

22. Esta herramienta podría hacer que disminuya la carga de trabajo de un diseñador *

Marca solo un óvalo.

	1	2	3	4	5	
Tota	☐	☐	☐	☐	☐	Totalmente de acuerdo

Opiniones

En esta sección es libre de expresar cualquier opinión respecto a los BTs generados o la herramienta.

23. Respuesta libre

Este contenido no ha sido creado ni aprobado por Google.

Google Formularios

Figura D.4: Formulario de recogida de datos demográficos (cuarta parte)

Glosario

Abstract Factory	El patrón Abstract Factory es un patrón de programación que consiste en mantener una clase que permita crear instancias de objetos relacionados gracias al polimorfismo.
AI	Artificial Intelligence
Almacén vectorial	Un almacén vectorial (o vector store) es una base de datos especializada en el almacenaje de la información en forma de vectores (o embeddings), la cual permite recuperar los datos por similitud matemática a otros vectores.
API	Application Programming Interface
Behavior Tree	Un Behavior Tree es un modelo de ejecución de tareas de forma jerárquica basado en árboles usada en el campo de la robótica y la inteligencia artificial
Blackboard	La Blackboard es la memoria compartida usada en un Behavior Tree. Ésta funciona como un diccionario de pares clave-valor, que pueden ser variables de entrada/salida.
CSV	Comma Separated Values
Embedding	Un embedding es un medio para representar la información compleja de forma numérica. Esto se consigue trasladando la información a un espacio vectorial en el que a cada unidad se le determina unas coordenadas según el contexto que tiene a su alrededor. Así, conceptos cercanos se mantienen matemáticamente cerca y son más sencillos de relacionar.

FAISS	FAISS (Facebook AI Similarity Search) es una librería de Meta que permite la búsqueda y el agrupamiento eficiente de vectores densos con el fin de facilitar la búsqueda por similitud.
Fine-tuning	El Fine-tuning es una técnica de Machine Learning que consiste en reentrenar ciertas partes de modelos ya preentrenados con nuevos datos con el objetivo de especializarlo en un tema concreto.
IA	Inteligencia Artificial
LLM	Un LLM (Large Language Model) es un modelo de aprendizaje automático profundo (deep learning) que ha sido entrenado con grandes cantidades de texto con propósito general de predicción de palabras.
LSTM	Long Short-Term Memory
Motor de videojuegos	Un motor de videojuegos es un software compuesto por varias librerías (motor de renderizado, motor de físicas, motor de audio, etc.) que permiten el desarrollo y la ejecución de un videojuego.
NPC	Non Playable Character
Ollama	Ollama es una aplicación basada en el motor de inferencia “llama.cpp” que permite descargar y ejecutar modelos localmente.
RAG	RAG (Retrieval Augmented Generation) es una técnica de Inteligencia Artificial que permite mejorar las respuestas de modelos de lenguaje buscando información en documentos externos que tengan que ver con la pregunta del usuario.
RNN	Recurrent Neural Network
RPG	Rol-Playing Game
Singleton	El patrón Singleton es un patrón de programación que consiste en garantizar que una clase tenga una instancia única y que se pueda acceder globalmente a ésta.